

CS620T: Compilers - Principles and Implementation

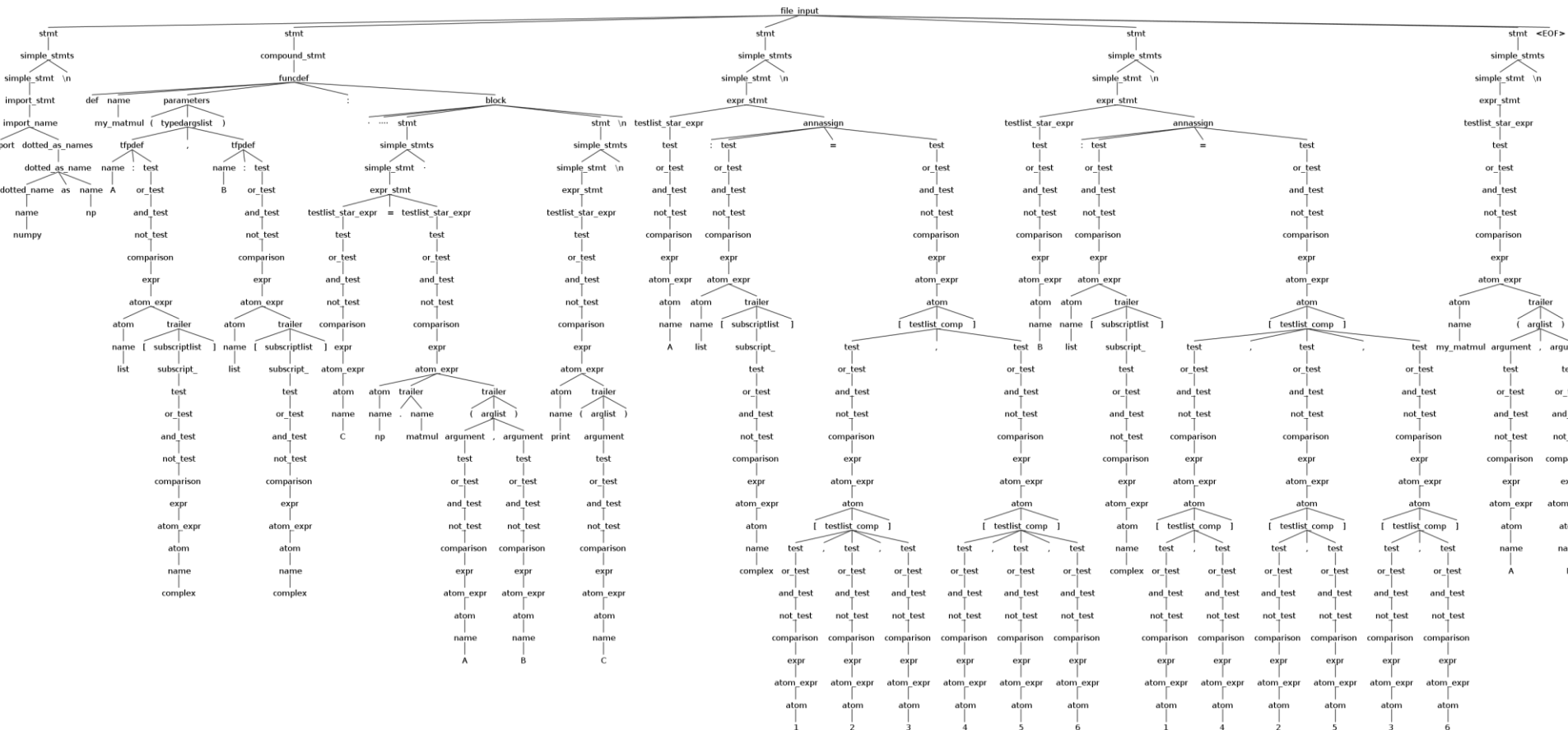
Autumn 2026

Week 5: Semantic processing (contd.) : AST
construction, Symbol Table construction, and
Intermediate Representation

Viewing ParseTree for a Python program using ANTLR

Refer to the README in the github repository

Parse tree for matmul_complex.py



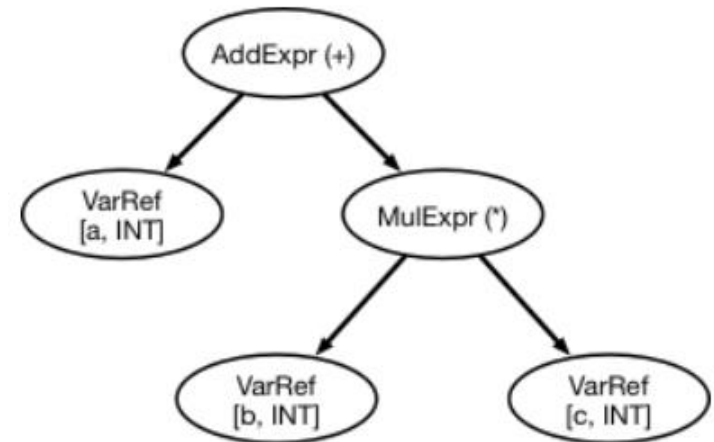
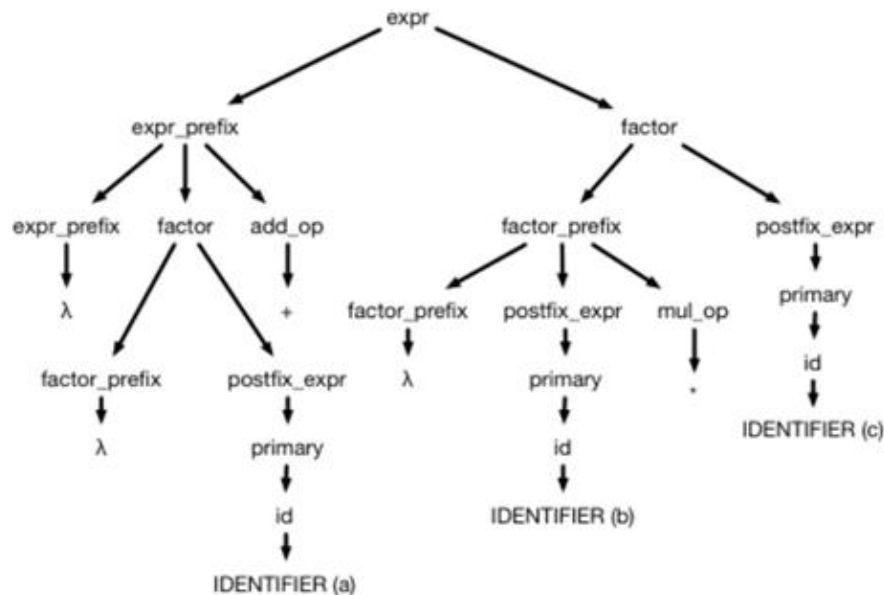
Compare this with the AST that you printed on the terminal (or visualized in the last class)

Clang AST

Clang

- C language family *frontend* for LLVM
- One of the responsibilities of the frontend is:
 - Reading a program text and producing an AST (abstract syntax tree), which is a representation of the program.

Parse Tree vs. AST (a*b+c)



- Clang AST – enables you to develop tools
 - for analysing program text

Clang Usage

- As Clang Plugin
 - E.g. `clang -fpass-plugin=mypass.so test.c`
(a silly use case: replacing + operator in `test.c` with *)
- LibTooling
 - A c++ program (with main function) written to do some analysis of some other program (C, C++)
 - E.g. `bin/myanalyzer test.c`
- LibClang
 - E.g., if We want to write a python program that analyses C++ source code

Clang AST

- Is rich (has source code location info at every node)
- Has types resolved
 - A design choice of compiler construction is that type checking can be done separately from AST construction
- Is huge
 - ~10K .cpp files
 - > 100k lines of code
- Has several classes, objects of which form the AST nodes.
 - The AST nodes are optimized for size.

Clang AST Classes

- `ASTContext` – one of the first classes that we encounter
AST nodes point to e.g.:
Identifier Table - storing identifiers
Source Manager – managing source code locations
- organizes information around the AST
- provides entry point to the AST with the help of
`TranslationUnitDecl* getTranslationUnitDecl()`

Clang AST Core Classes

- Decl

- Base class for many other classes e.g. VarDecl

`int x=100; //int x corresponds to a VarDecl node`

exercise: what type of node represents 100? IntegerLiteral

Other examples: CXXRecordDecl

- Stmt

- CompoundStmt

No common base class



ReturnStmt -> e.g., return x;

- BinaryOperator -> e.g., `i > 0`, `x + 2`

- Is an expression (Expr) and expressions are Stmts

- Type

- PointerType
- ParenType

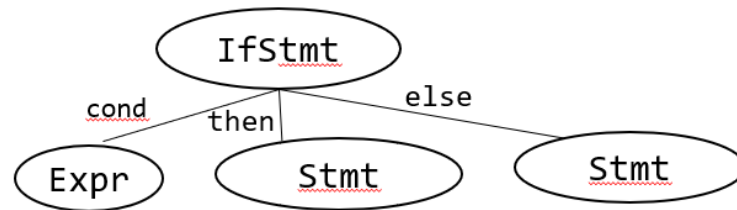
Clang AST Glue Classes

- DeclContext
 - Inherited by Decl's that contain other Decl's
- TemplateArgument
 - To represent template arguments
- NestedNameSpecifier
- QualType
 - For representing type qualifiers e.g. const, unsigned etc.

Clang AST Glue Methods

Help in traversing AST

- IfStmt : getThen(), getElse(), getCond()



Traverse across parts of AST

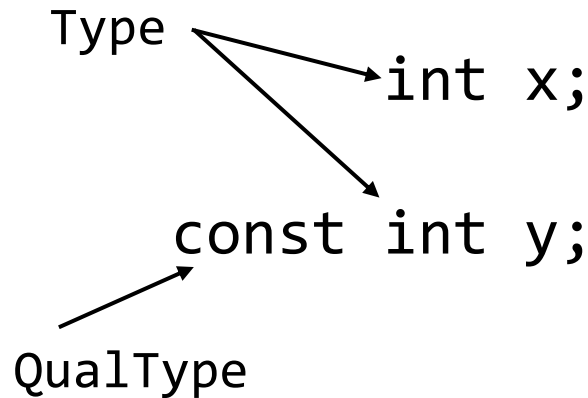
- CXXRecordDecl: getDescribedClassTemplate() method
- Type: getAsCXXRecordDecl() method

For getting the declaration of a struct/union/class

Detour: declaration vs. definition of function definitions, variable definitions, class/struct definitions in C/C++

Types

- Represent types in AST

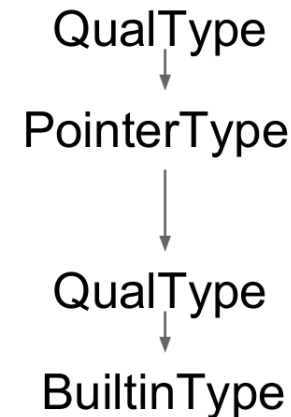


`int * const * y;`

Types can be nested. Qualifier is sandwiched.

Exercise: what is the type of y? Constant pointer to a `int *`

```
class PointerType{
    QualType getPointeeType() const;
}
```



Navigating Source

- `SourceLocation` – tells the location of a token

AST Traversal

- Multiple abstractions available:
 - RecursiveASTVisitor
 - ASTMatcher
- Next: RecursiveASTVisitor

ASTFrontendAction

```
class FindNamedClassAction : public clang::ASTFrontendAction {  
public:  
    virtual std::unique_ptr<clang::ASTConsumer> CreateASTConsumer(  
        clang::CompilerInstance &Compiler, llvm::StringRef InFile) {  
        return  
std::make_unique<FindNamedClassConsumer>(&Compiler.getASTContext());  
    }  
};
```

- Inherit from `ASTFrontendAction` and override `CreateASTConsumer`
 - `ASTFrontendAction` is an interface that allows user-specific actions to happen during compilation.
- `CreateASTConsumer`
 - Consumes the AST produced by the Clang parser

ASTConsumer

```
class FindNamedClassConsumer : public clang::ASTConsumer {
public:
    explicit FindNamedClassConsumer(ASTContext *Context)
        : visitor(Context) {}

    virtual void HandleTranslationUnit(clang::ASTContext &Context) {
        visitor.TraverseDecl(Context.getTranslationUnitDecl());
    }
private:
    FindNamedClassVisitor visitor;
};
```

- Override (as many) methods to take user-specific action on visiting AST nodes.
- `HandleTranslationUnit` called after entire source code is parsed (not while it is being parsed)
 - `ASTContext` class represents AST for the source file.
 - `Visitor.TraverseDecl(Context.getTranslationUnitDecl())` begins visiting nodes of the tree

RecursiveASTVisitor

```
class FindNamedClassVisitor
: public RecursiveASTVisitor<FindNamedClassVisitor> {
public:
    bool VisitCXXRecordDecl(CXXRecordDecl *Declaration) {
        //for illustration only. Dump shows which nodes already visited.
        Declaration->dump();

        // The return value indicates whether we want the traversal to proceed.
        // Return false to stop the traversal of the AST.
        return true;
    }
};
```

- Don't call Visit functions directly
- Implement VisitStmt, VisitDecl, VisitPantry etc. as per your needs

Pattern Matching and Source Location

```
if (Declaration->getQualifiedNameAsString() == "n::m::C") {  
    FullSourceLoc FullLocation = Context->getFullLoc(Declaration->getBeginLoc());  
    if (FullLocation.isValid())  
        llvm::outs() << "Found declaration at "  
            << FullLocation.getSpellingLineNumber() << ":"  
            << FullLocation.getSpellingColumnNumber() << "\n";  
}
```

- Get the location manager from ASTContext

Symbol Table

- *A symbol table* maintains
 - Symbolic names
 - Attributes of a name
 - E.g. type, scope, accessibility
- Used to manage declarations of symbols and their correct usage

Symbol Table – Names

For the sample program shown below identify all names (note: this is not a valid micro program)

```
PROGRAM scope_test
BEGIN
#global declarations
FUNCTION void f(float, float, float)
FUNCTION void g(int)
{
    INT w, x;
    {
        FLOAT x, z;
        f(x, w, z);
    }
    g(x);
}

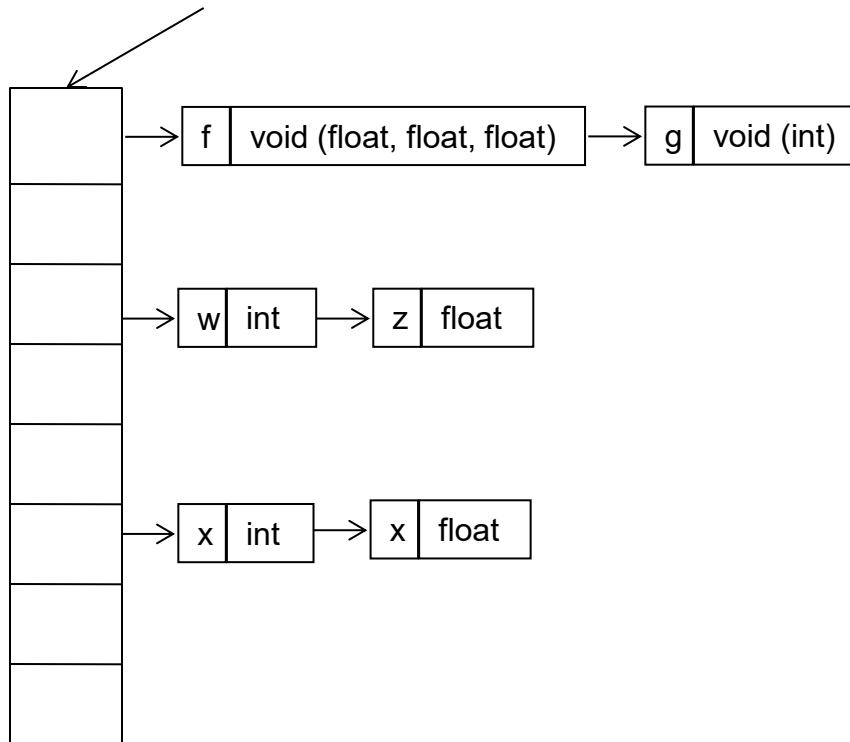
END
```

Symbol Table Implementation – High-level Requirements

- Should accommodate:
 - Efficient retrieval of names
 - Frequent insertion and deletion of names
- Should consider *scopes*

Symbol Table – an implementation

Hash table of names



```
PROGRAM scope_test
BEGIN
#global declarations
FUNCTION void f(float, float, float)
FUNCTION void g(int)
{
    INT w, x;
    {
        FLOAT x, z;
        f(x, w, z);
    }
    g(x);
}
END
```

Symbol Table – an implementation

Hash table of names

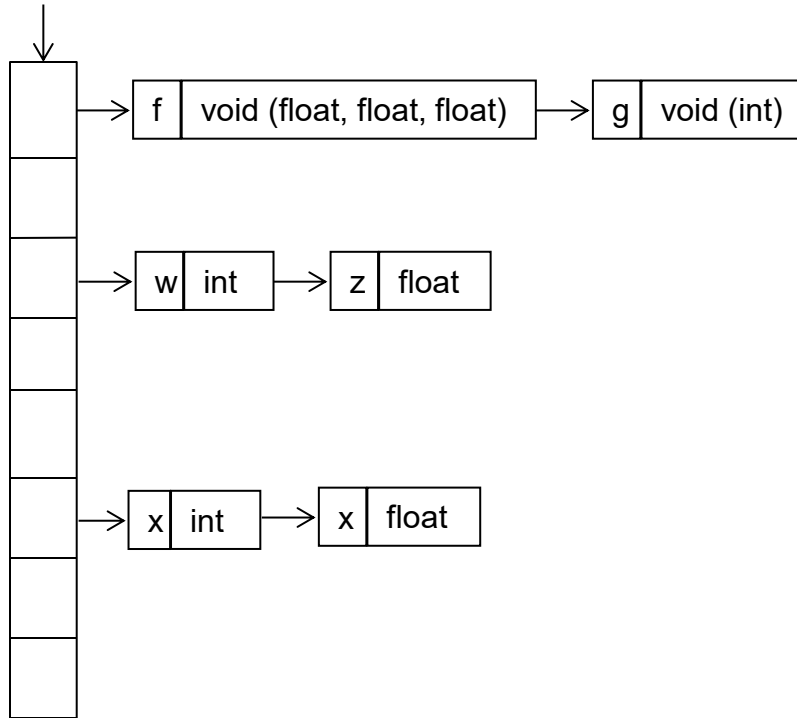
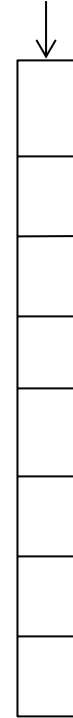


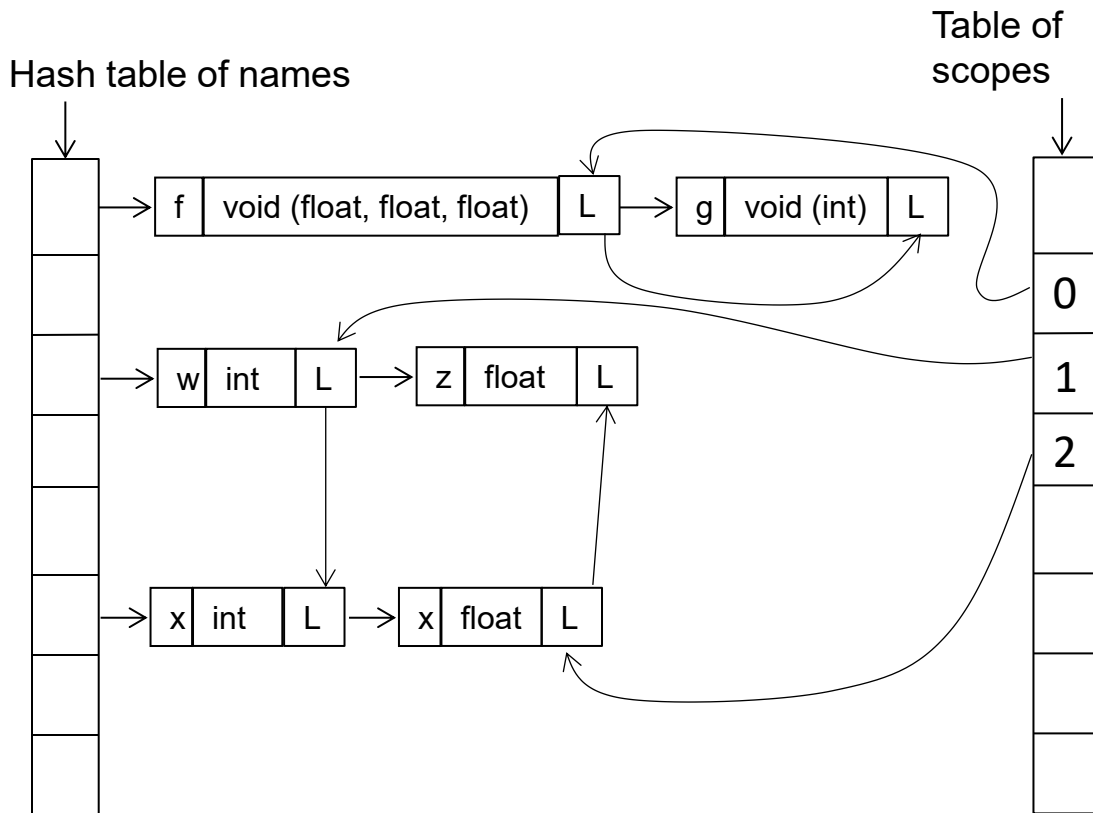
Table of scopes



```
PROGRAM scope_test
BEGIN
#global declarations
FUNCTION void f(float, float, float)
FUNCTION void g(int)
{
    INT w, x;
    {
        FLOAT x, z;
        f(x, w, z);
    }
    g(x);
}
END
```

- be aware of current scope
- Be aware of all active scopes
- Chain names by their scope-levels

Symbol Table – an implementation

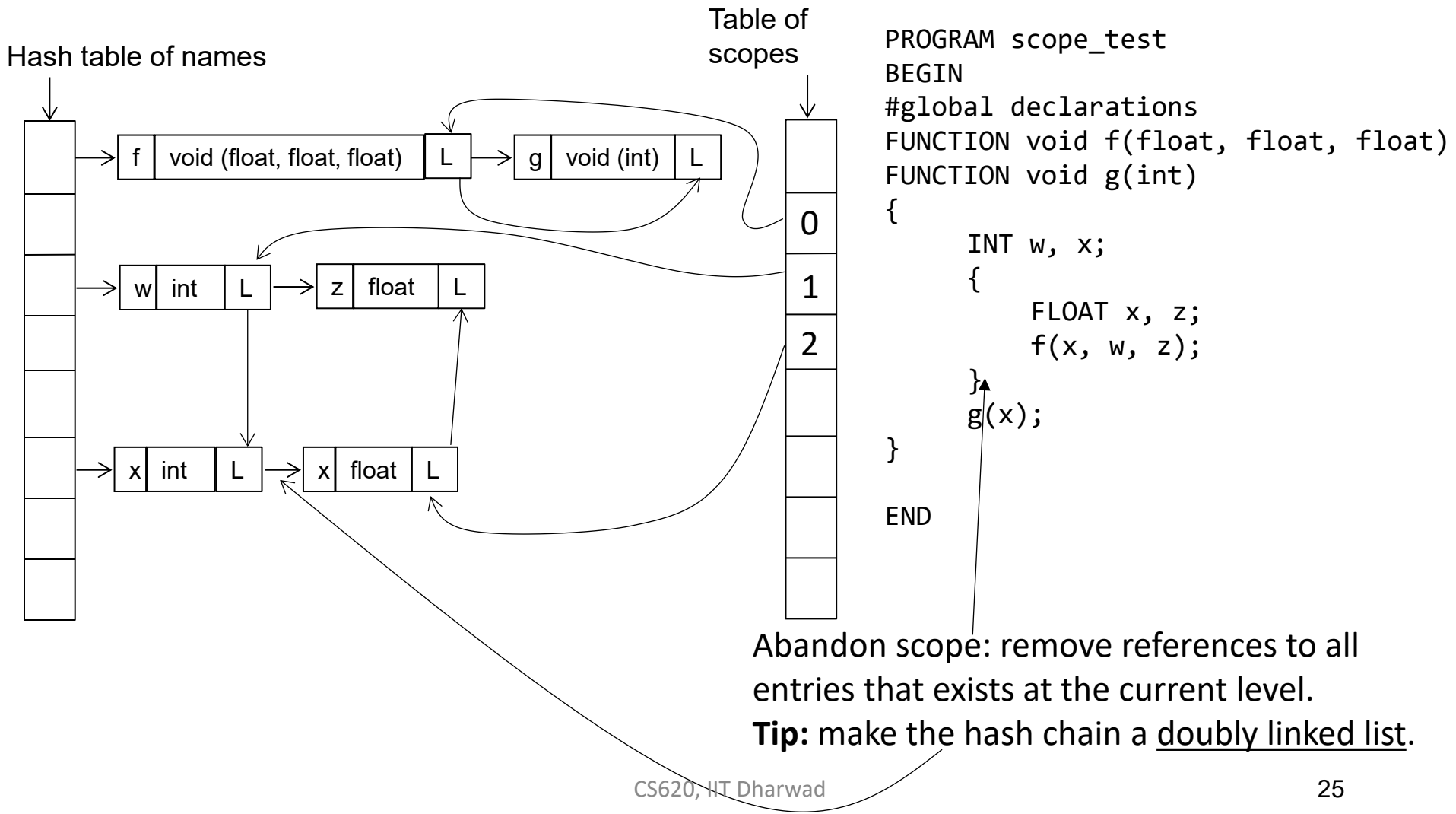


```

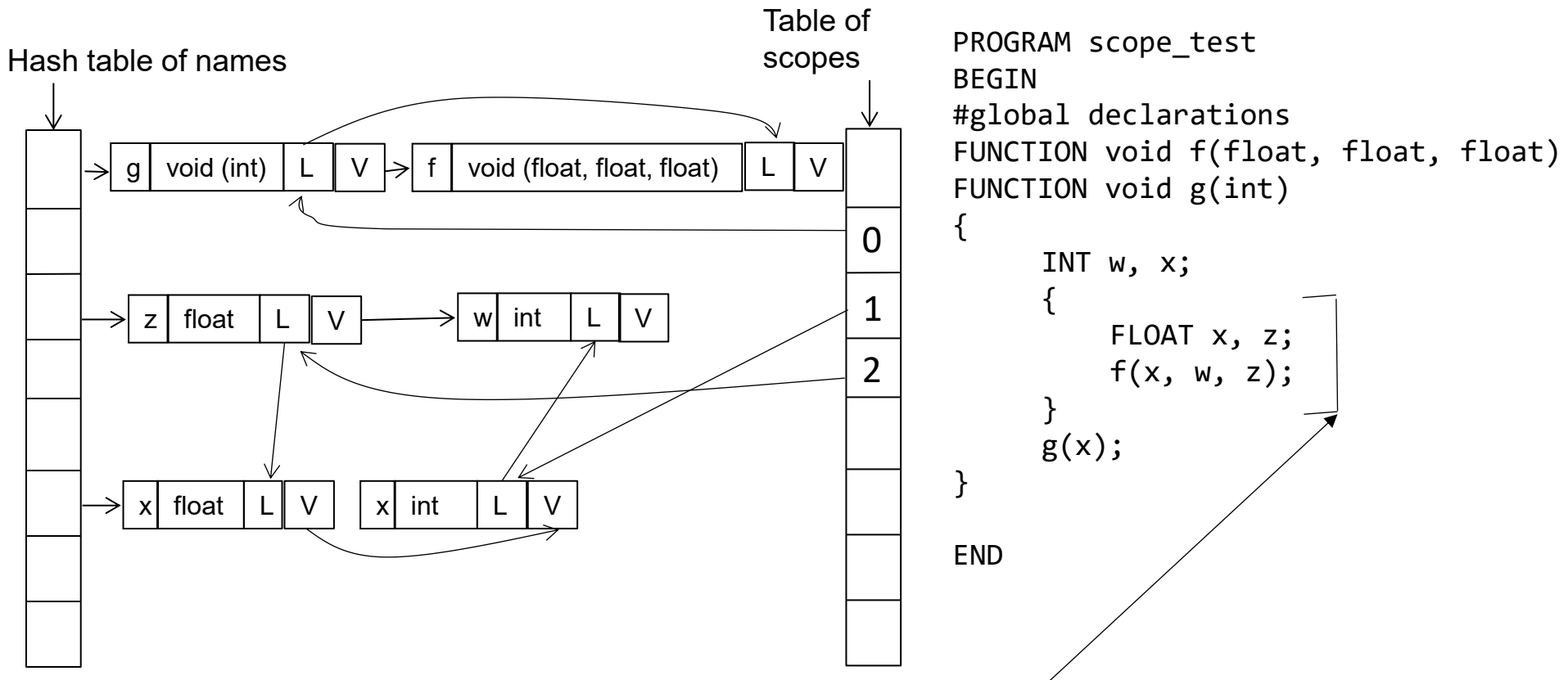
PROGRAM scope_test
BEGIN
#global declarations
FUNCTION void f(float, float, float)
FUNCTION void g(int)
{
    INT w, x;
    {
        FLOAT x, z;
        f(x, w, z);
    }
    g(x);
}
END
  
```

- Chain names by their scope-levels

Symbol Table – an implementation



Symbol Table – an implementation

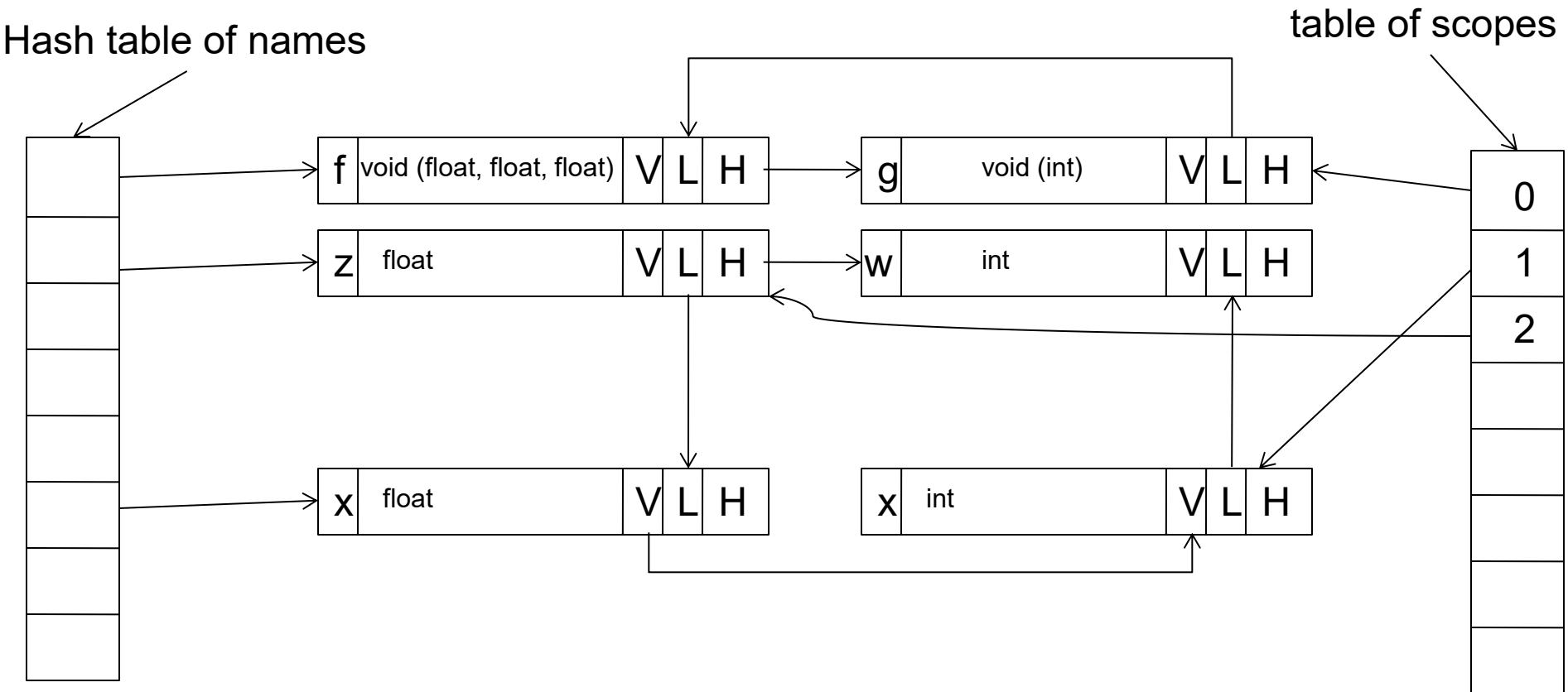


Notice the order of objects: “insert at the front of the list”

What if I want to access the integer x here?

Tip: maintain an ordered stack for each symbol name appearing in the current scope.

Symbol Table – an implementation



Intermediate Representation

- Compilers need to synthesize code based on the ‘interpretation’ of the syntactic structure
- Code can be generated with the help of AST or can directly do it in semantic actions (recall: SDTs augment grammar rules with program fragments. Program fragments contain semantic actions.)
- Generated code can be directly executed on the machine or an intermediate form such as 3-address code can be produced.

3 Address Code (3AC)

- **What is it?** sequence of elementary program instructions
 - Linear in structure (no hierarchy) unlike AST
 - Format:
`op A, B, C` //means $C = A \text{ op } B$.
//op: ADDI, MULI, SUBF, DIVF, GOTO, STOREF etc.
 - E.g.

program text

3-address code

```
INT x;  
FLOAT y, z;  
z:=x+y;
```

```
ADDF x y T1  
STOREF T1 z
```

```
INT a, b, c, d;  
d = a-b/c;
```

```
DIVI b c T1  
SUBI a T1 T2  
STOREF T2 d
```

Comments:

d = a-b/c; is broken into:
t1 = b/c;
t2 = a-t1;
d = t2;

3 Address Code (3AC)

- **Why is it needed?** To perform *significant* optimizations such as:
 - common sub-expression elimination
 - statically analyze possible values that a variable can take etc.

How?

Break the long sequence of instructions into “basic blocks” and operate on/analyze a graph of basic blocks

3 Address Code (3AC)

- **How is it generated?** Choices available:

1. Do a post-order walk of AST

- Generate/Emit code as a string/data_object (seen later) when you visit a node
- Pass the code to the parent node

Parent generates code for self after the code for children is generated. The generated code is appended to code passed by children and passed up the tree

```
data_object generate_code() {  
    //preprocessing code  
    data_object lcode=left.generate_code();  
    data_object rcode=right.generate_code();  
    return generate_self(lcode, rcode);  
}
```

2. Can generate directly in semantic routines or after building AST

3 Address Code (3AC)

- Generating 3AC directly in semantic routines.



```
INT x;  
x:=3*4+5+6+7;
```

```
MULI 3 4 T1  
ADDI T1 5 T2  
ADDI T2 6 T3  
ADDI T3 7 T4  
STOREI T4 x
```

Comments:

$x = 3*4+5+6+7$ is broken into:
 $t1 = 3*4;$
 $t2 = 5+t1;$
 $t3 = 6+t2;$
 $t4 = 7+t3;$
 $x = t4$

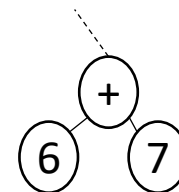
- Walk the AST in post-order and infer at an internal node (labelled op) that it computes a constant expression



```
INT x;  
x:=3*4+5+6+7;
```

```
STOREI 30 x
```

Comments:



Simple cases

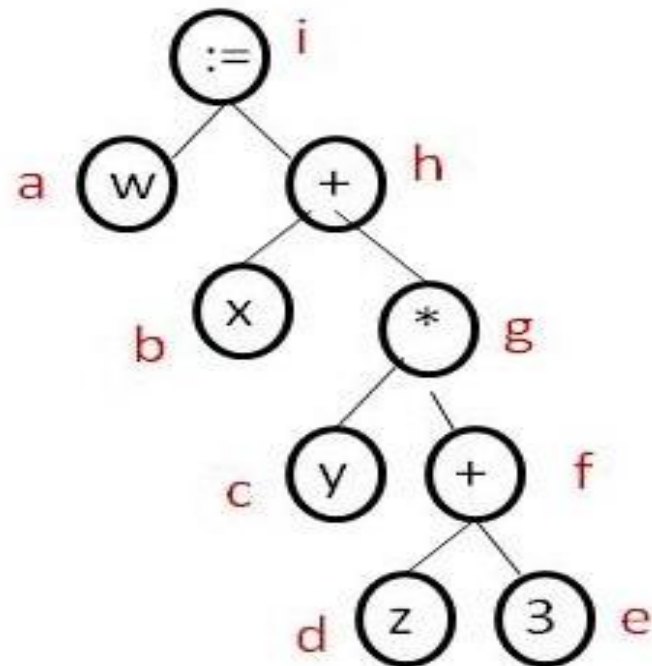
- Generating code for constants/literals
 - Store constant in temporary
 - Optional: pass up flag specifying this is a constant
- Generating code for identifiers
 - Generated code depends on whether identifier is used as L-value or R-value
 - Is this an address? Or data?
 - One solution: just pass identifier up to next level
 - Mark it as an L-value (it's not yet data!)
 - Generate code once we see how variable is used

Generating code for expressions

- Create a new temporary for result of expression
- Examine data-objects from subtrees
- If temporaries are L-values, load data from them into new temporaries
 - Generate code to perform operation
 - In project, no need to explicitly load (variables can be operands)
- If temporaries are constant, can perform operation immediately
 - No need to perform code generation!
- Store result in new temporary
 - Is this an L-value or an R-value?
- Return code for entire expression

Example - assignment statement

AST for $w := x + y * (z + 3);$ $\Rightarrow$



Visit Node a:

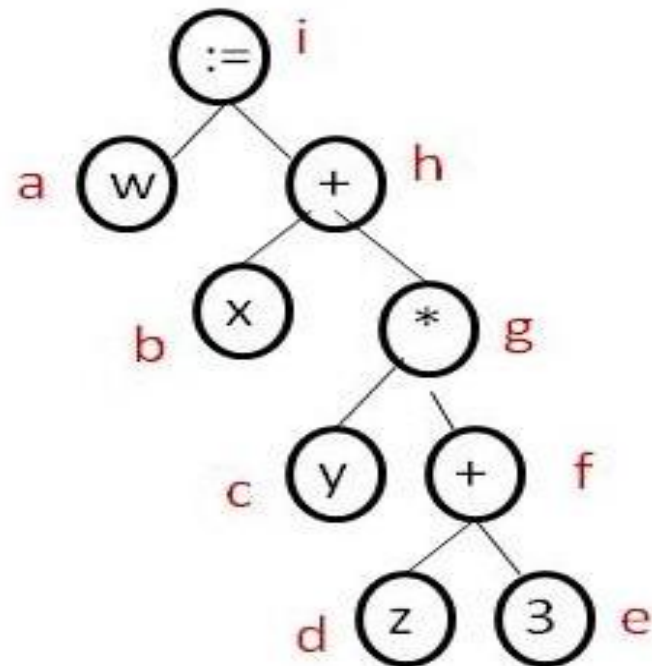
Temp: w

Type: l-value

Code: --

Example - assignment statement

AST for $w := x + y * (z + 3);$ $\Rightarrow$



Visit Node b:

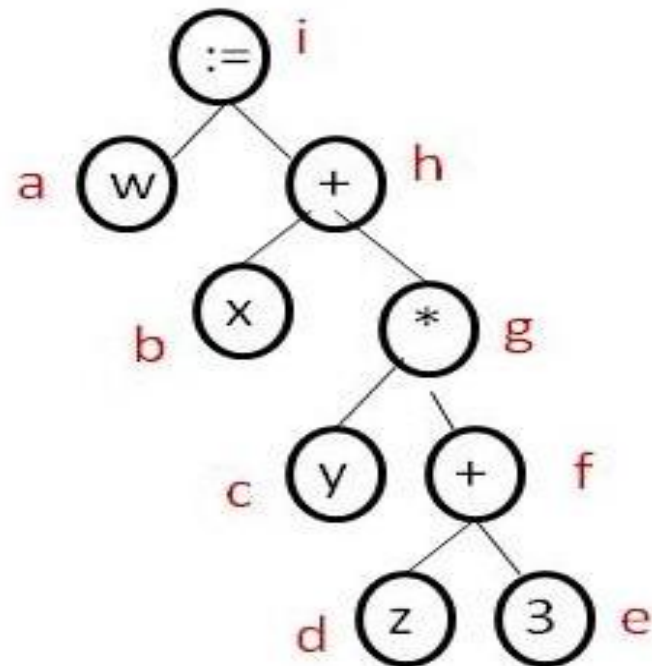
Temp: x

Type: l-value

Code: --

Example - assignment statement

AST for $w := x + y * (z + 3);$ $\Rightarrow$



Visit Node c:

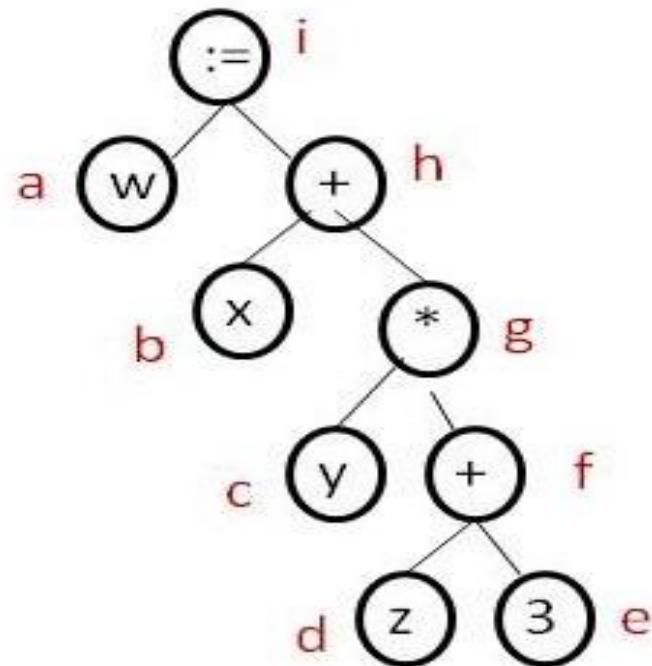
Temp: y

Type: l-value

Code: --

Example - assignment statement

AST for $w := x + y * (z + 3);$ $\Rightarrow$



Visit Node d:

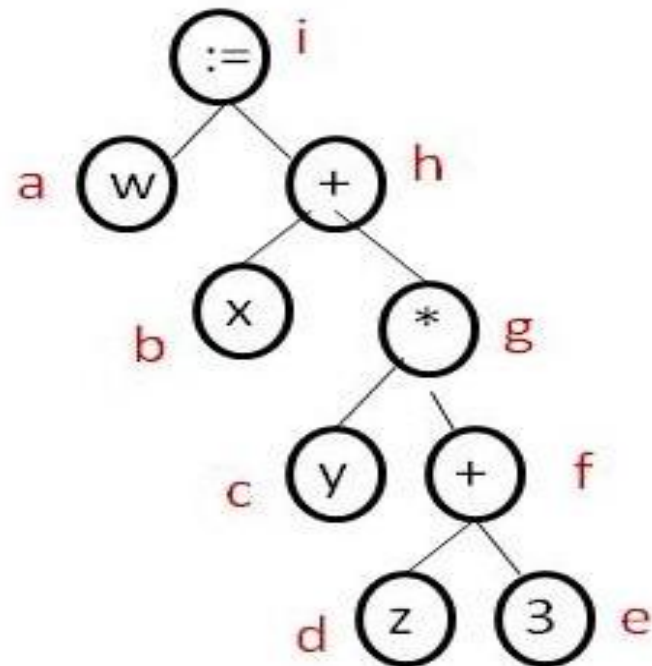
Temp: z

Type: l-value

Code: --

Example - assignment statement

AST for $w := x + y * (z + 3);$ $\Rightarrow$



Visit Node e:

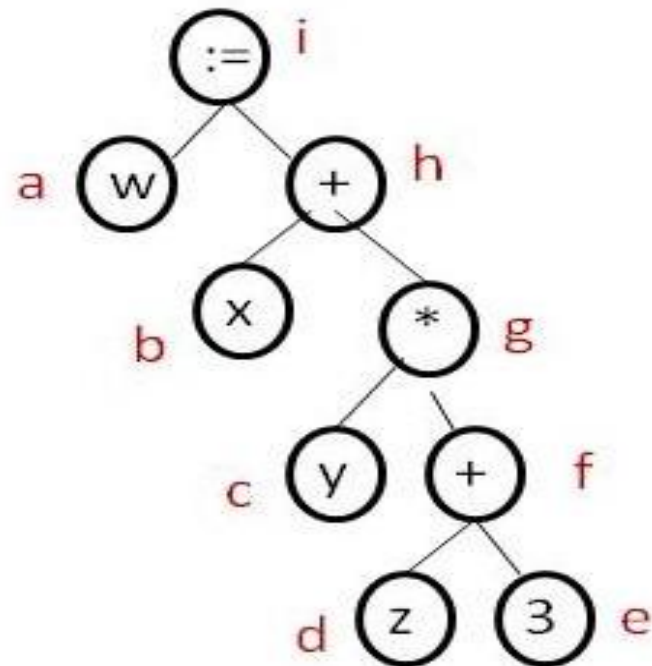
Temp: 3

Type: constant

Code: --

Example - assignment statement

AST for $w := x + y * (z + 3);$ $\Rightarrow$



Visit Node f:

Temp: T1

Type: R-value

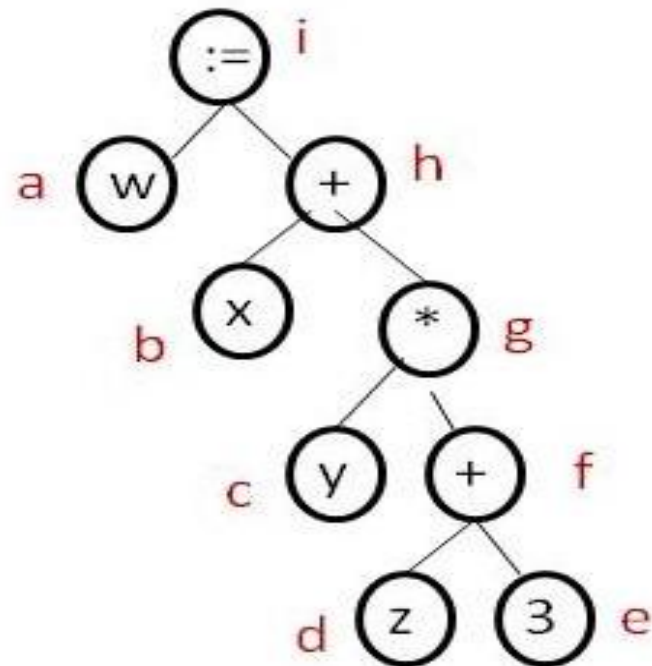
Code:

LD z T2

ADD T2 3 T1

Example - assignment statement

AST for $w := x + y * (z + 3);$ $\Rightarrow$



Visit Node g :

Temp: T3

Type: R-value

Code:

LD y T4

LD z T2

ADD T2 3 T1

MUL T4 T1 T3

Example - assignment statement

AST for $w := x + y * (z + 3);$ $\Rightarrow$

Visit Node h:

Temp: T5

Type: R-value

Code:

LD x T6

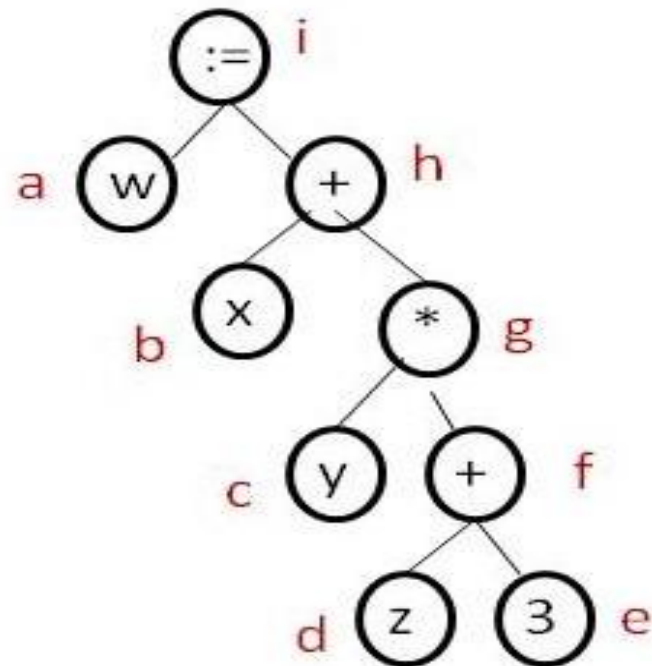
LD y T4

LD z T2

ADD T2 3 T1

MUL T4 T1 T3

ADD T6 T3 T5



Example - assignment statement

AST for $w := x + y * (z + 3);$ $\Rightarrow$

Visit Node i:

Temp: NA

Type: NA

Code:

LD x T6

LD y T4

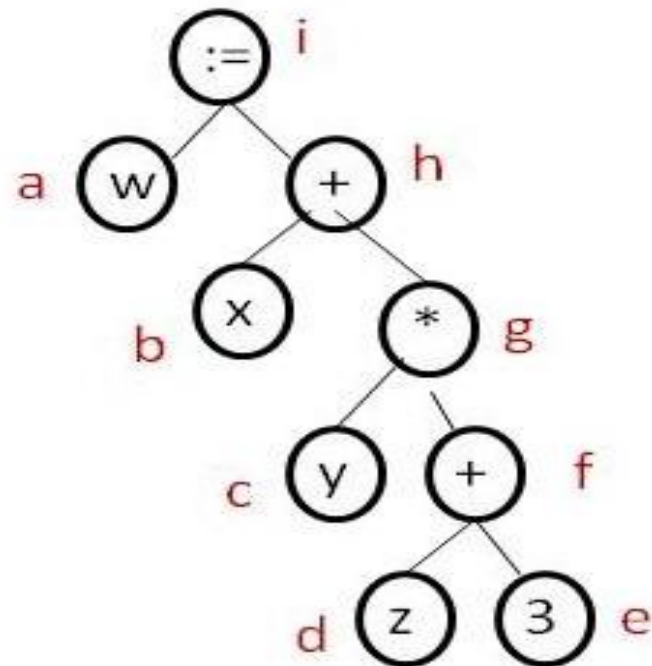
LD z T2

ADD T2 3 T1

MUL T4 T1 T3

ADD T6 T3 T5

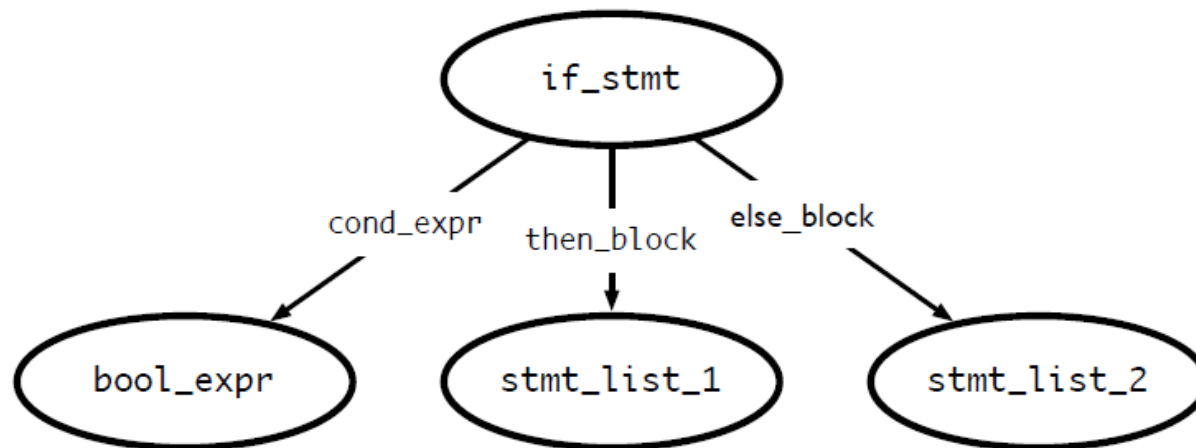
ST T5 w



If statements

```
if <bool_expr_1>  
    <stmt_list_1>  
else  
    <stmt_list_2>  
endif
```

If statements



Generating code for ifs

```
if <bool_expr_1>  
    <stmt_list_1>  
else  
    <stmt_list_2>  
endif
```

```
<code for bool_expr_1>  
j<!op> ELSE_1  
<code for stmt_list_1>  
jmp OUT_1  
ELSE_1:  
    <code for stmt_list_2>  
OUT_1:
```

Notes on code generation

- The `<op>` in `j<!op>` is dependent on the type of comparison you are doing in `<bool_expr>`
- When you generate JUMP instructions, you should also generate the appropriate LABELs
- Remember: labels have to be unique!

Code-generation – if-statement

Program text

3AC

INT a, b;

Code-generation – if-statement

Program text	3AC
INT a, b;	Make entries in the symbol table

Code-generation – if-statement

Program text

3AC

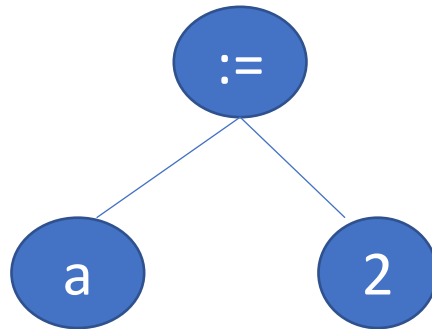
```
INT a, b;  
a := 2;
```

Code-generation – if-statement

Program text

3AC

```
INT a, b;  
a := 2;
```



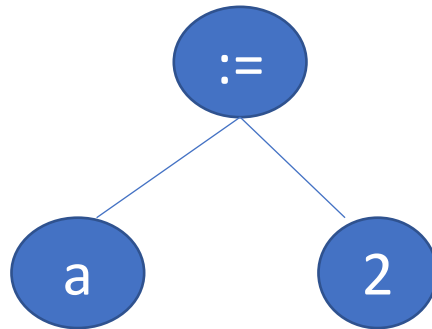
1. “a” is left-child, type=l-val. No code generated. *Return an object containing identifier details after verifying that “a” is present in the symbol table.*

Code-generation – if-statement

Program text

3AC

INT a, b;
a := 2;



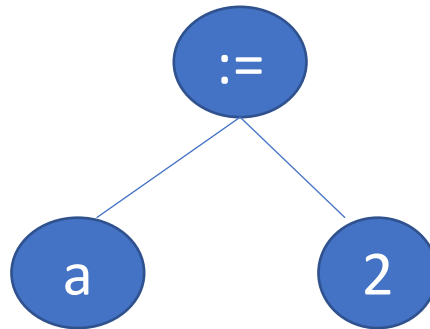
1. “a” is left-child, type=l-val. No code generated. Pass up the identifier.
2. “2” is right-child, type=const. No code generated.

Code-generation – if-statement

Program text

3AC

```
INT a, b;  
a := 2;
```



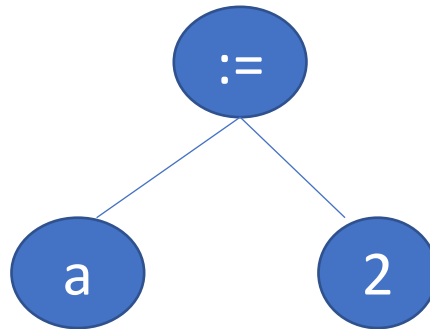
1. “a” is left-child, type=l-val. No code generated. Pass up the identifier.
2. “2” is right-child, type=const. No code generated.
3. Create a temporary T1 to store the result of the expression

Code-generation – if-statement

Program text

3AC

```
INT a, b;  
a := 2;
```

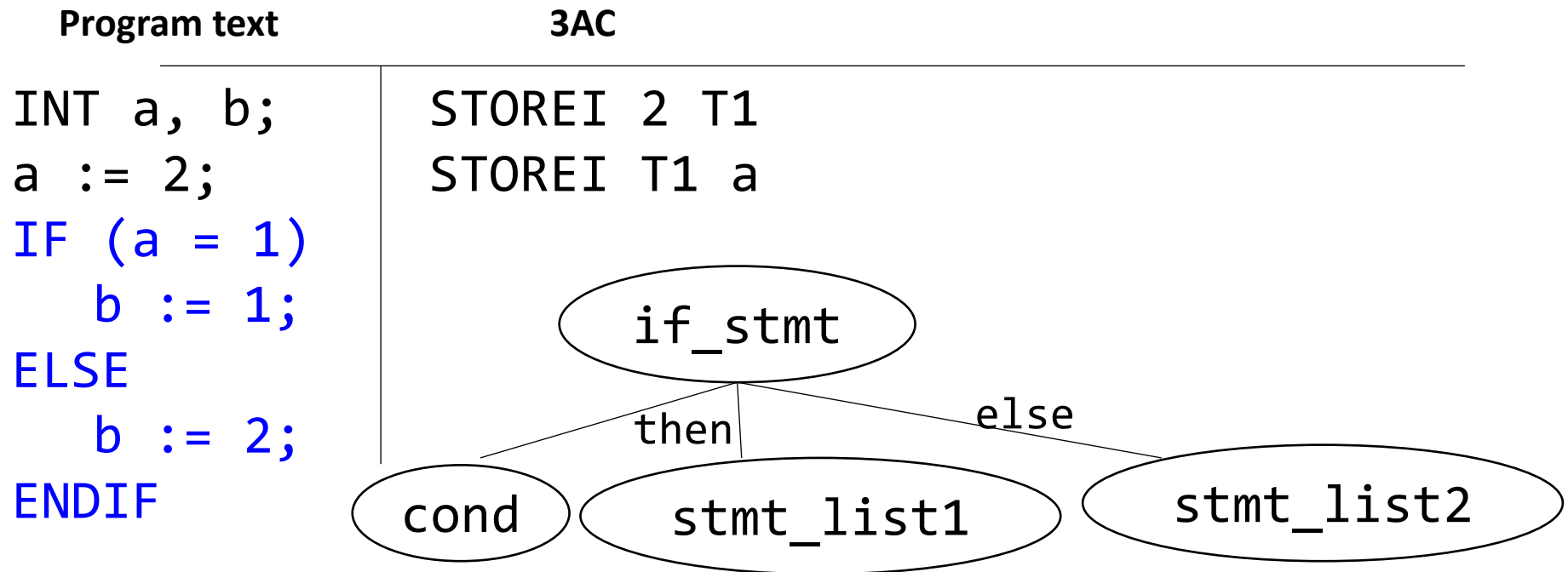


1. “a” is left-child, type=l-val. No code generated. Pass up the identifier.
2. “2” is right-child, type=const. No code generated.
3. Create a temporary T1 to store the result of the expression
 - Current node stores the op ‘:=’. A call to `process_op` stores the RHS data in LHS

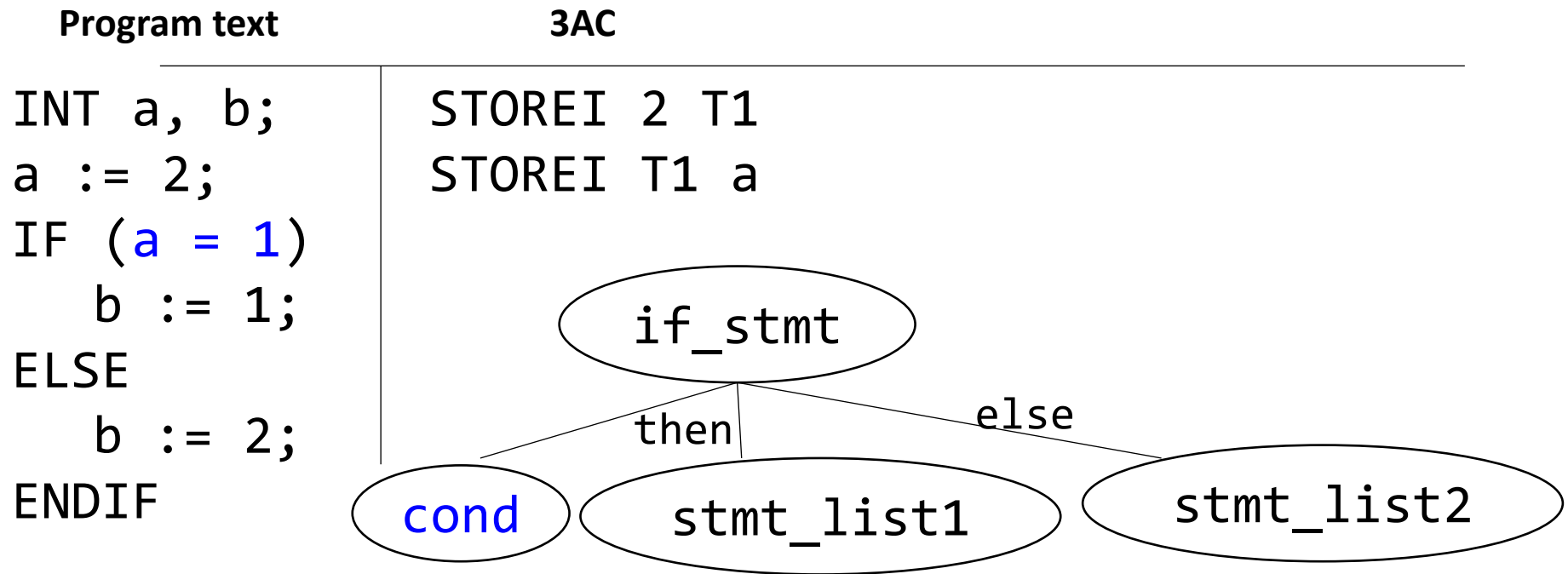
Code-generation – if-statement

Program text	3AC
INT a, b; a := 2;	STOREI 2 T1 STOREI T1 a

Code-generation – if-statement

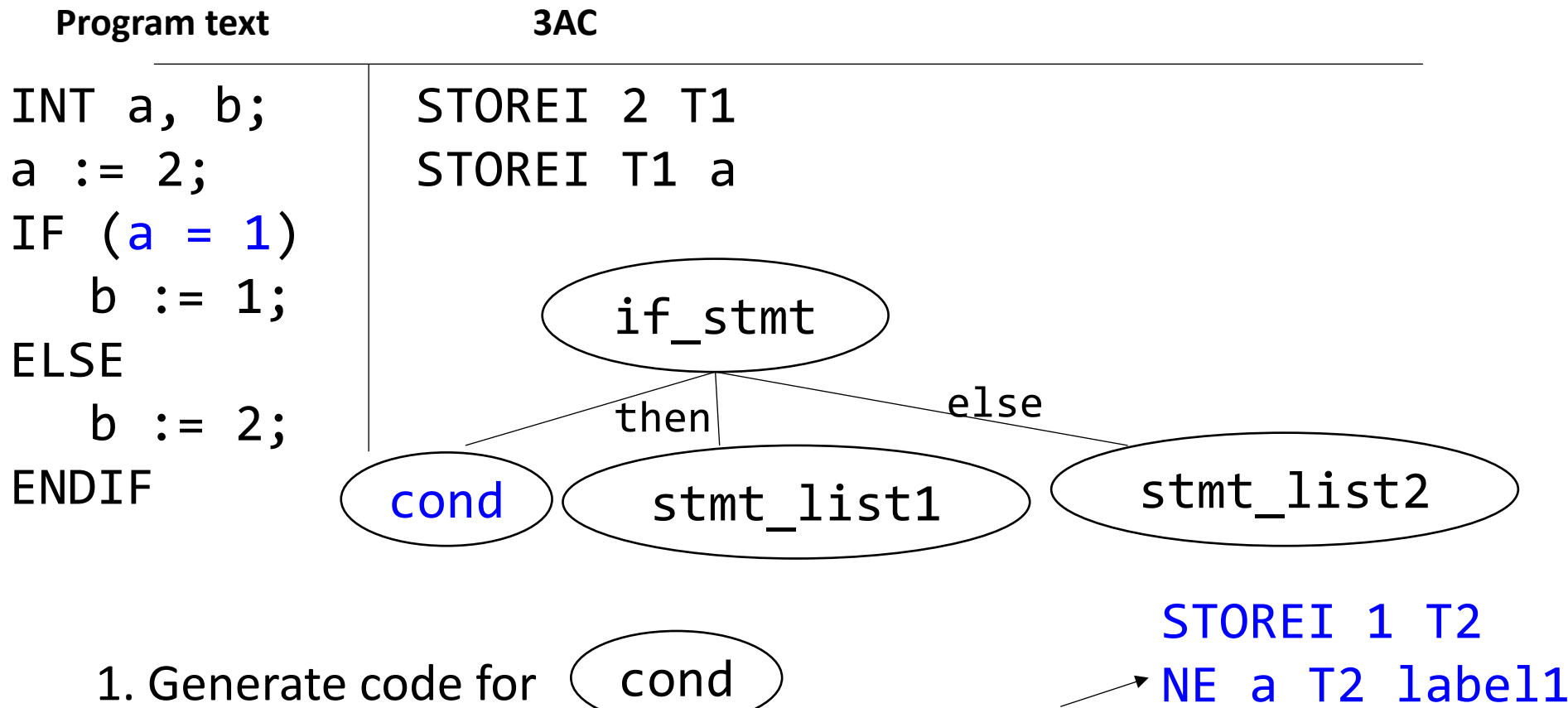


Code-generation – if-statement



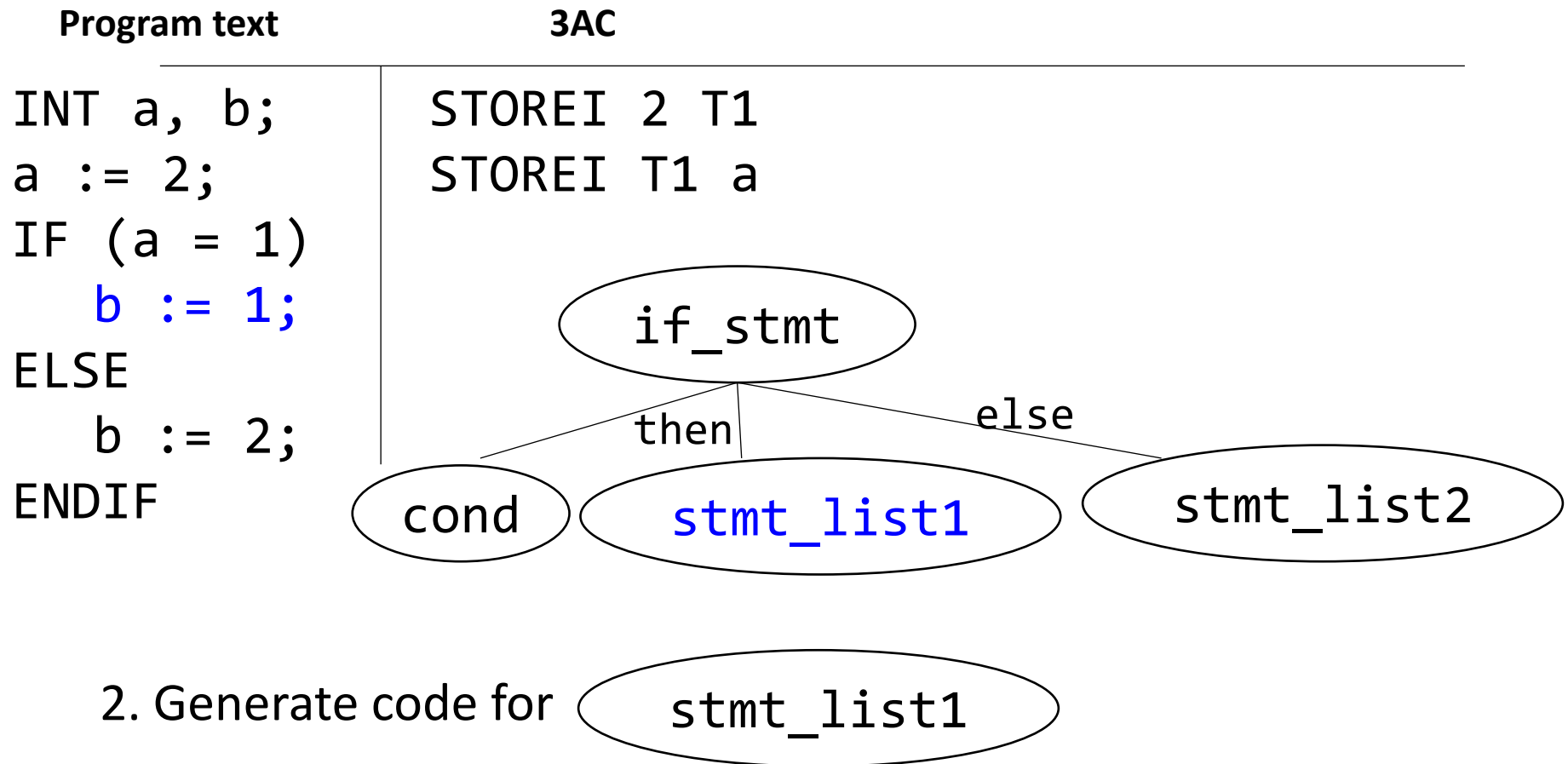
1. Generate code for **cond**

Code-generation – if-statement

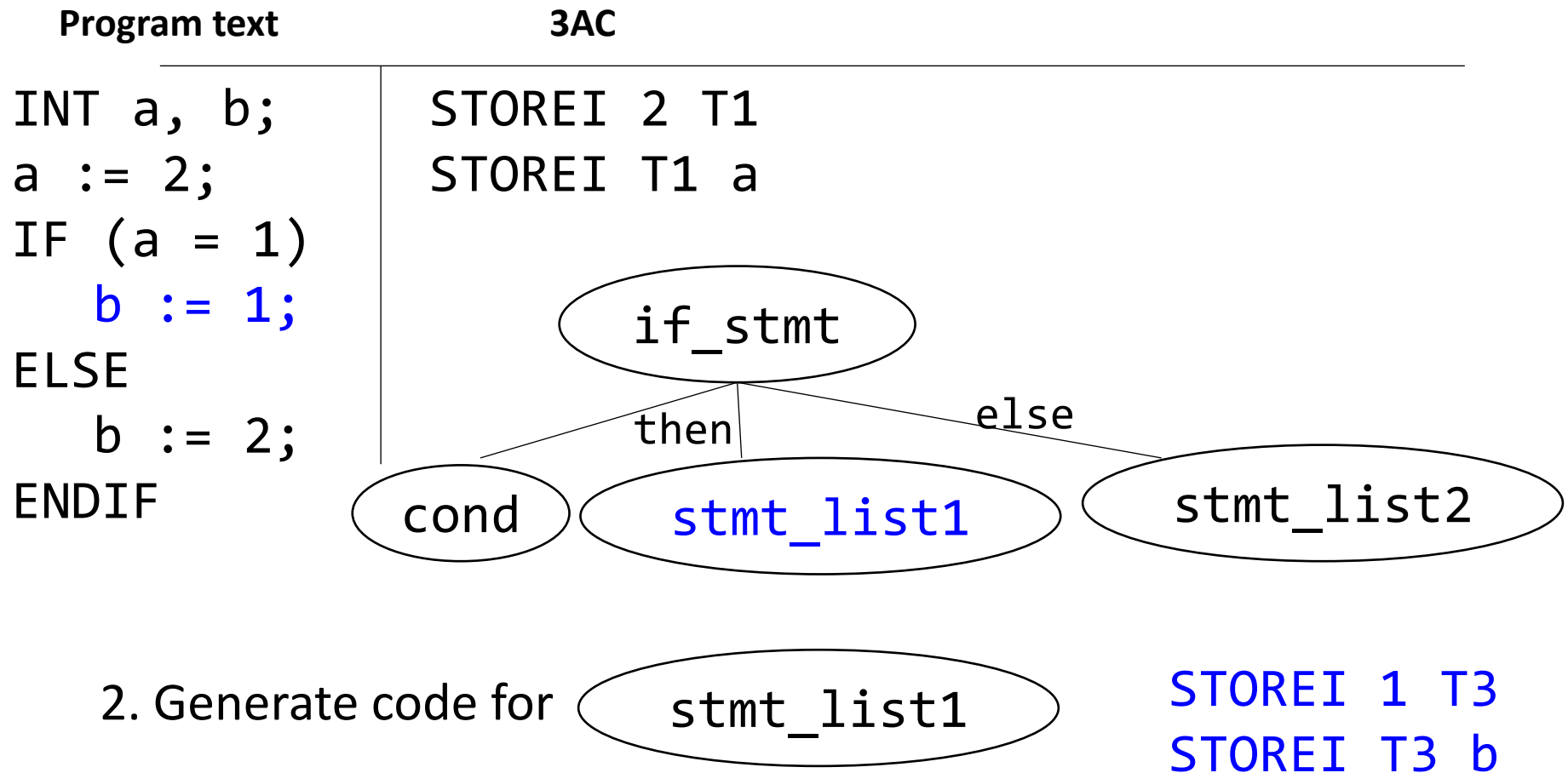


Note that to generate this instruction when cond node is visited, we need information about the label. This information can be passed on as a semantic record for the child node of the if construct. The record can be created by the IF construct (when the keyword IF is seen) and would be updated subsequently.

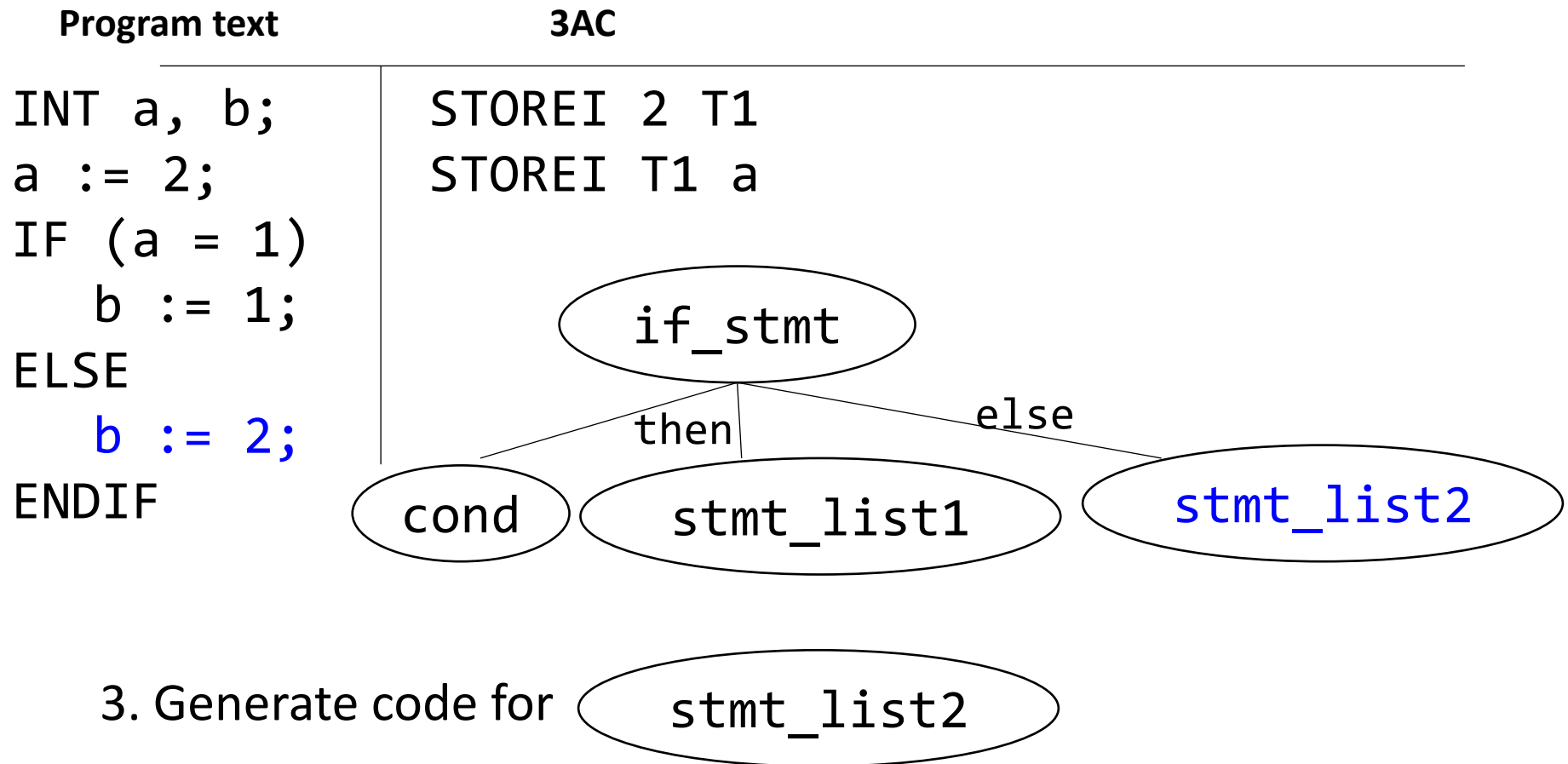
Code-generation – if-statement



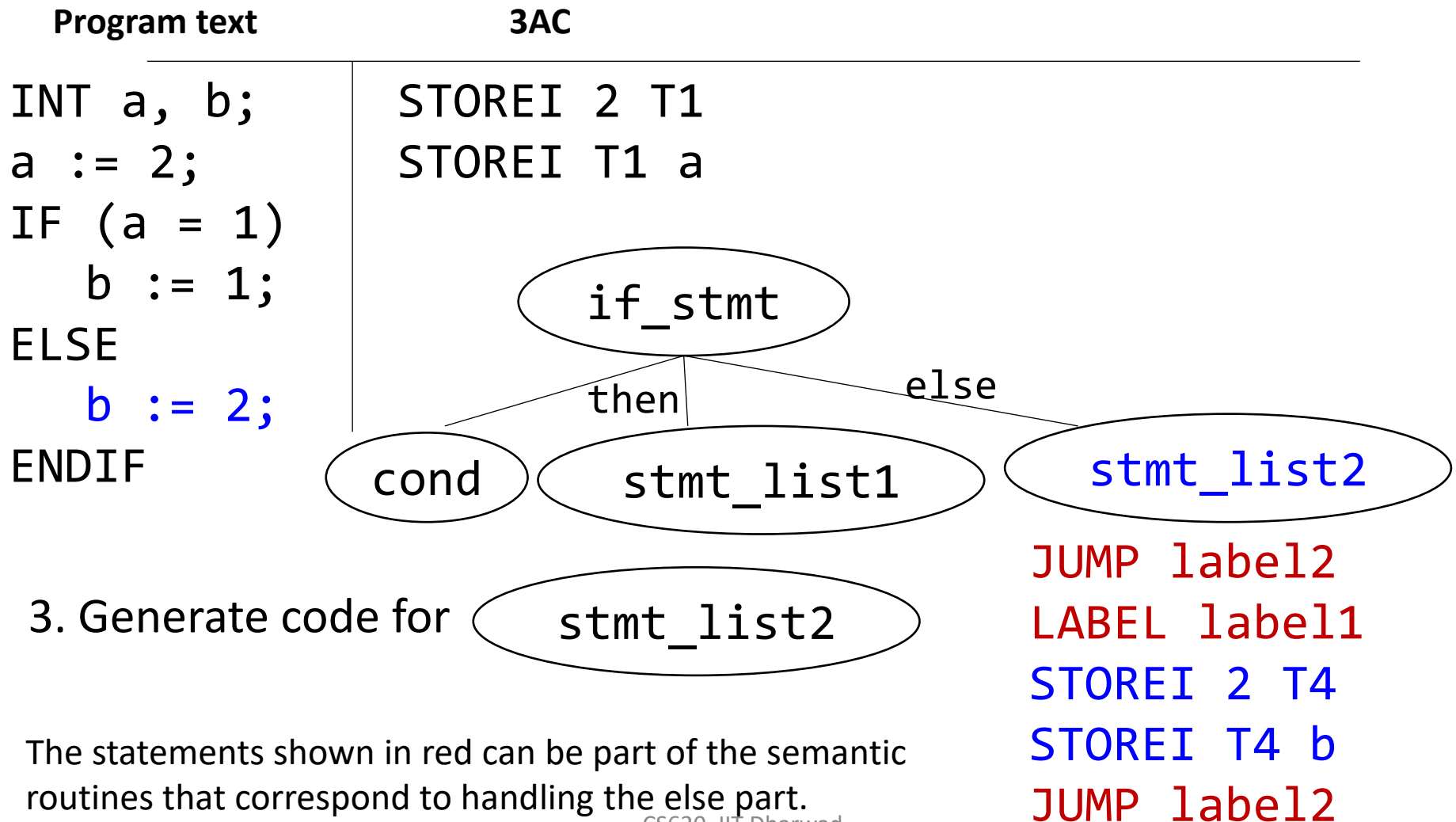
Code-generation – if-statement



Code-generation – if-statement

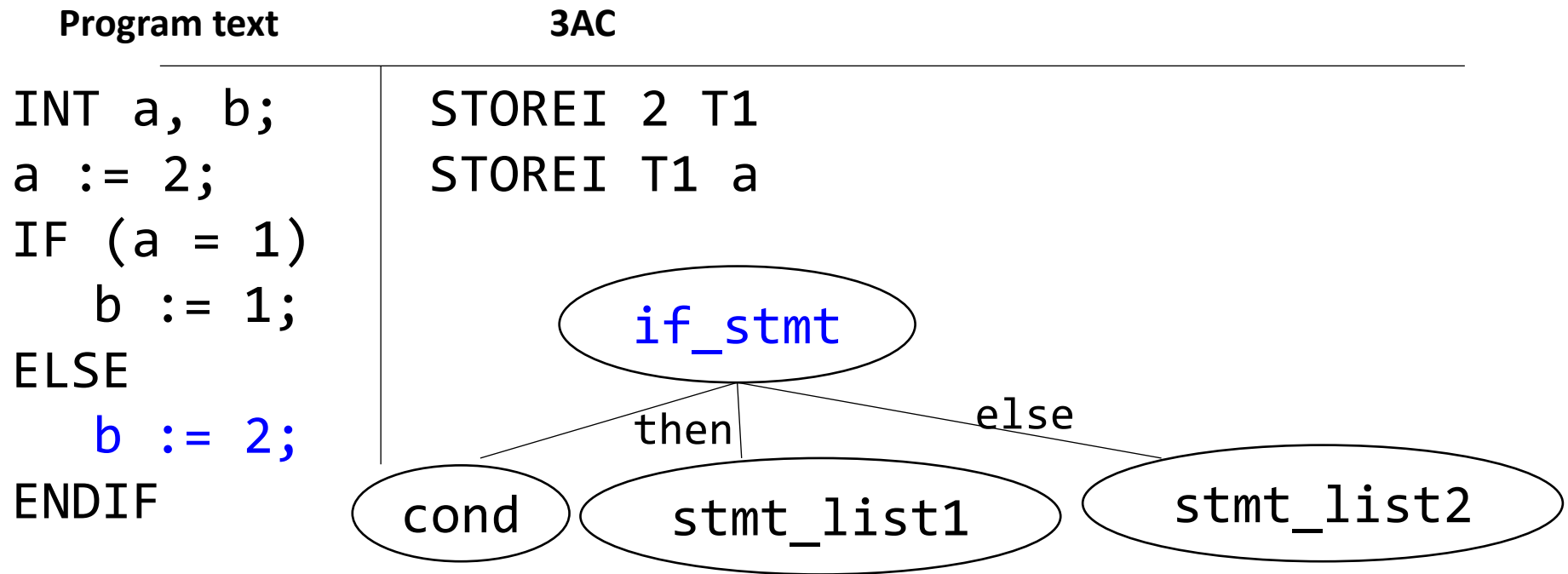


Code-generation – if-statement



The statements shown in red can be part of the semantic routines that correspond to handling the else part.

Code-generation – if-statement



3. Generate code for

if_stmt

LABEL label2

Code-generation – if-statement

Program text	3AC
INT a, b;	STOREI 2 T1 //a := 2
a := 2;	STOREI T1 a
IF (a = 1)	STOREI 1 T2 //a = 1?
b := 1;	NE a T2 label1
ELSE	STOREI 1 T3 //b := 1
b := 2;	STOREI T3 b
ENDIF	JUMP label2 //to out label
	LABEL label1 //else label begins here
	STOREI 2 T4 //b := 2
	STOREI T4 b
	JUMP label2 //jump to out label
	LABEL label2 //out label

Can also generate this code after seeing the token ENDIF (rather than as part of the routine that is executed when the whole production is matched)

Jumps and Labels?

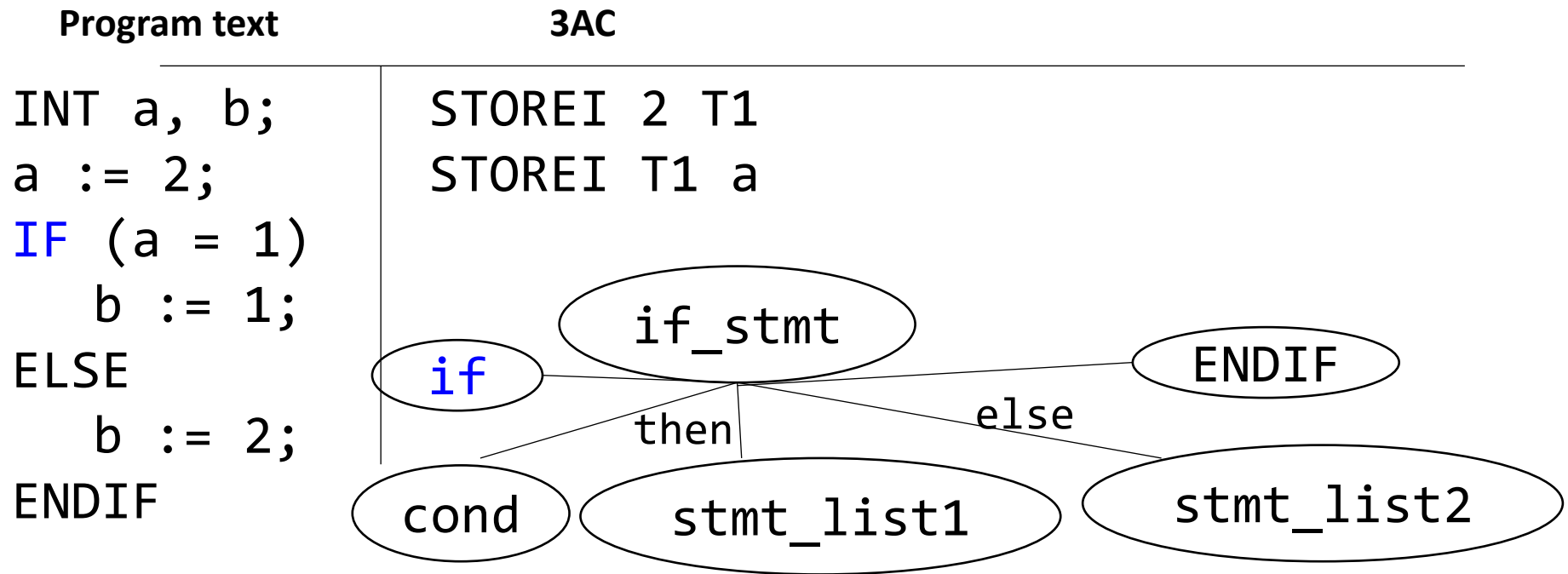
- Who will generate labels?
- When will the labels be generated?
- To what addresses will the labels be associated with?

How are targets of jumps decided?

If construct with semantic actions

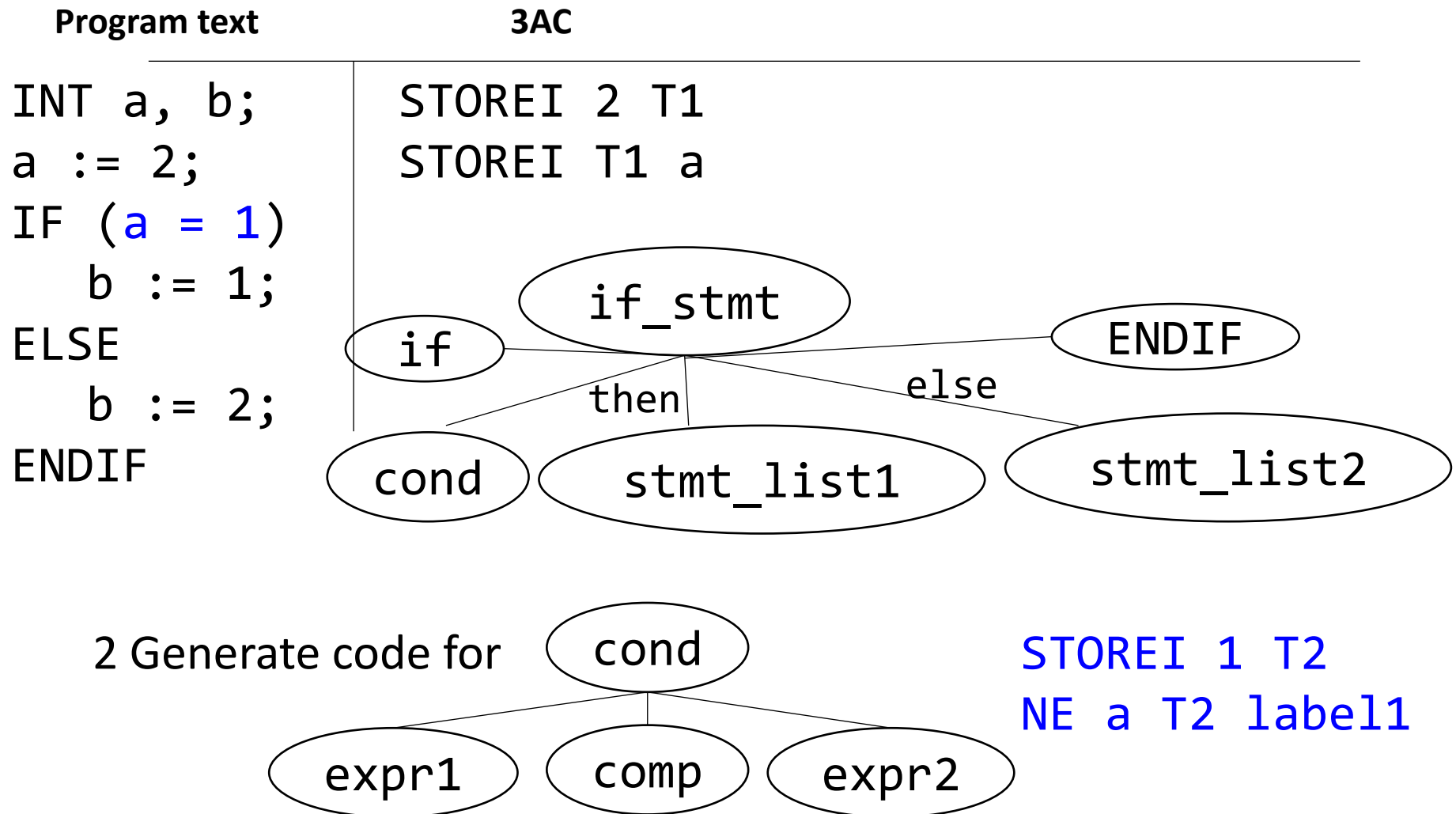
- If_stmt → if #start_if <b_expr> #testif then
 <stmt_list> <else_part> endif; #gen_out_label
- else_part → else #gen_jump #gen_else_label
 <stmt_list>

Code-generation – if-statement

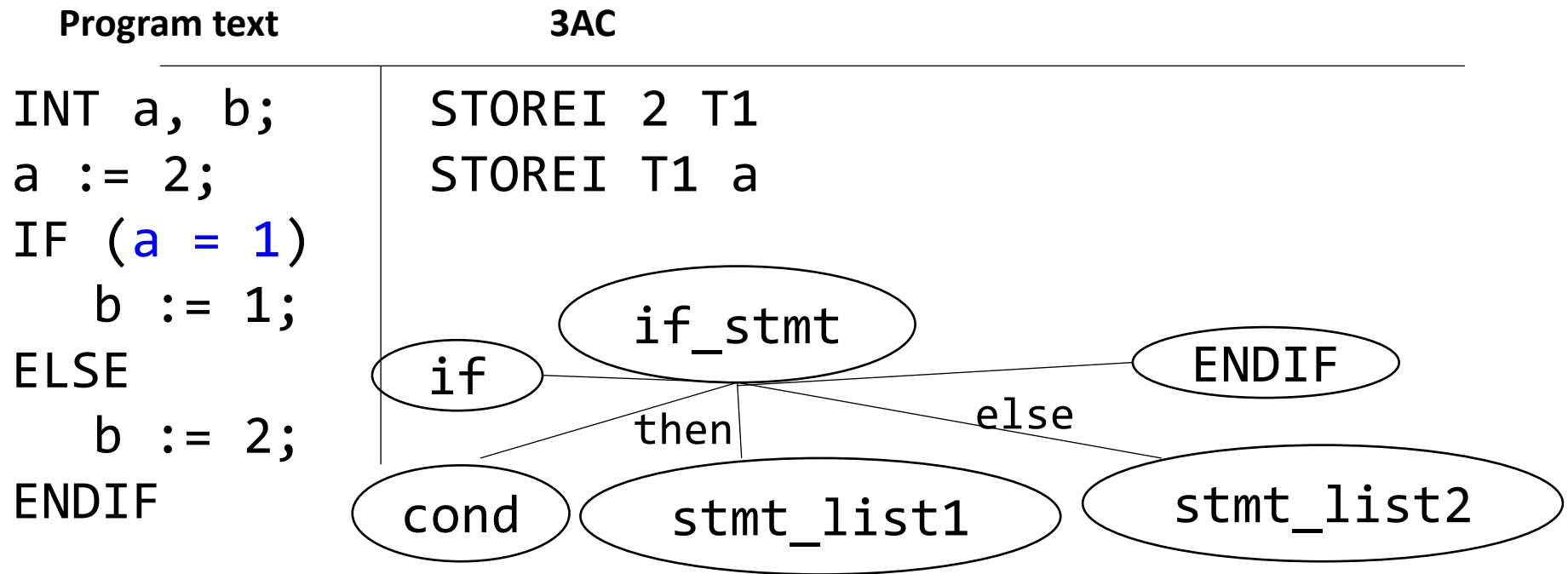


- 1 Generate out label and store it in semantic record of if_stmt (label11)
The #start_if routine is responsible for this

Code-generation – if-statement



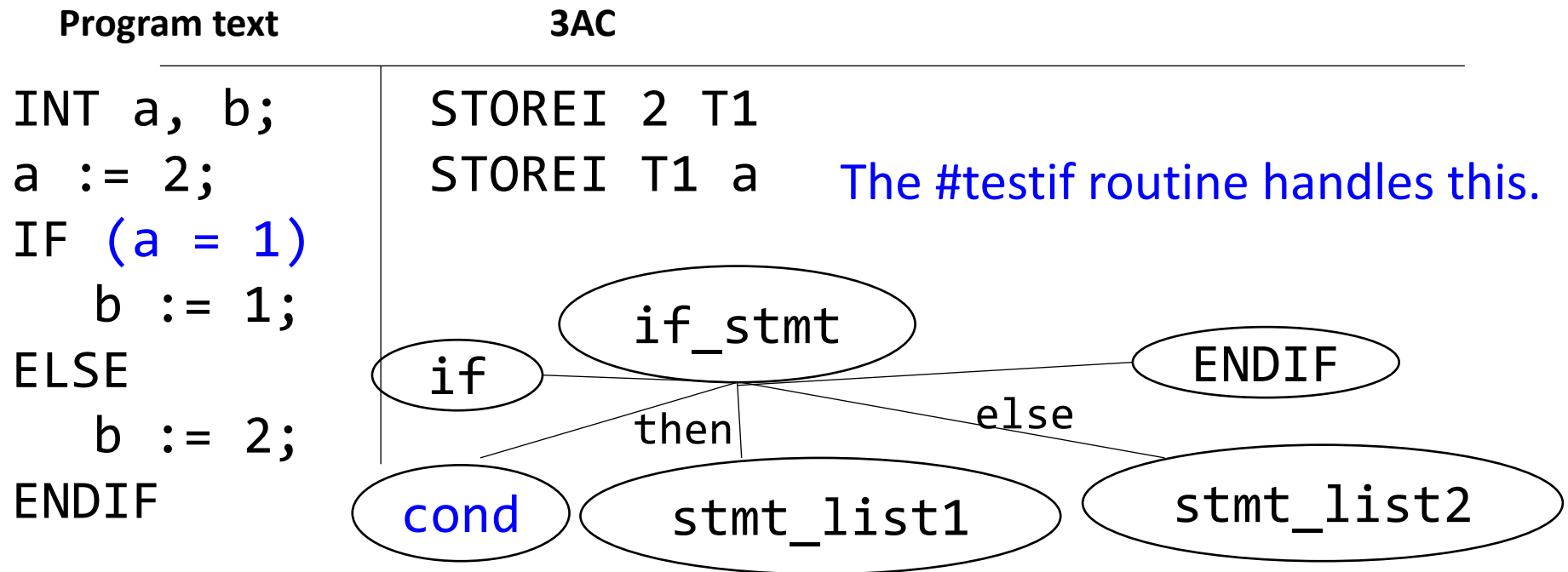
Code-generation – if-statement



2. Store the result of calling process_op, where op is "=", in the node cond (bool_expr1=false)

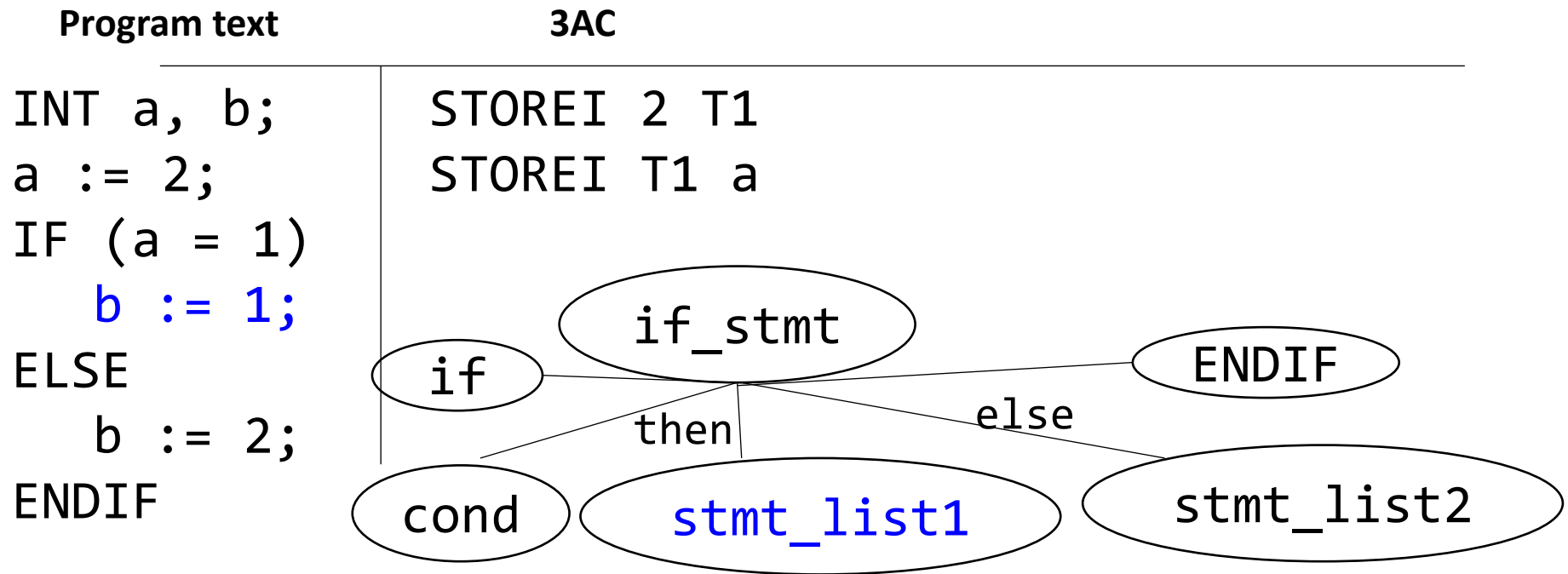
STOREI 1 T2
NE a T2 label1

Code-generation – if-statement



2. Create a label for the next else part (label 2). Generate statement: `JUMP0 T2 label2`

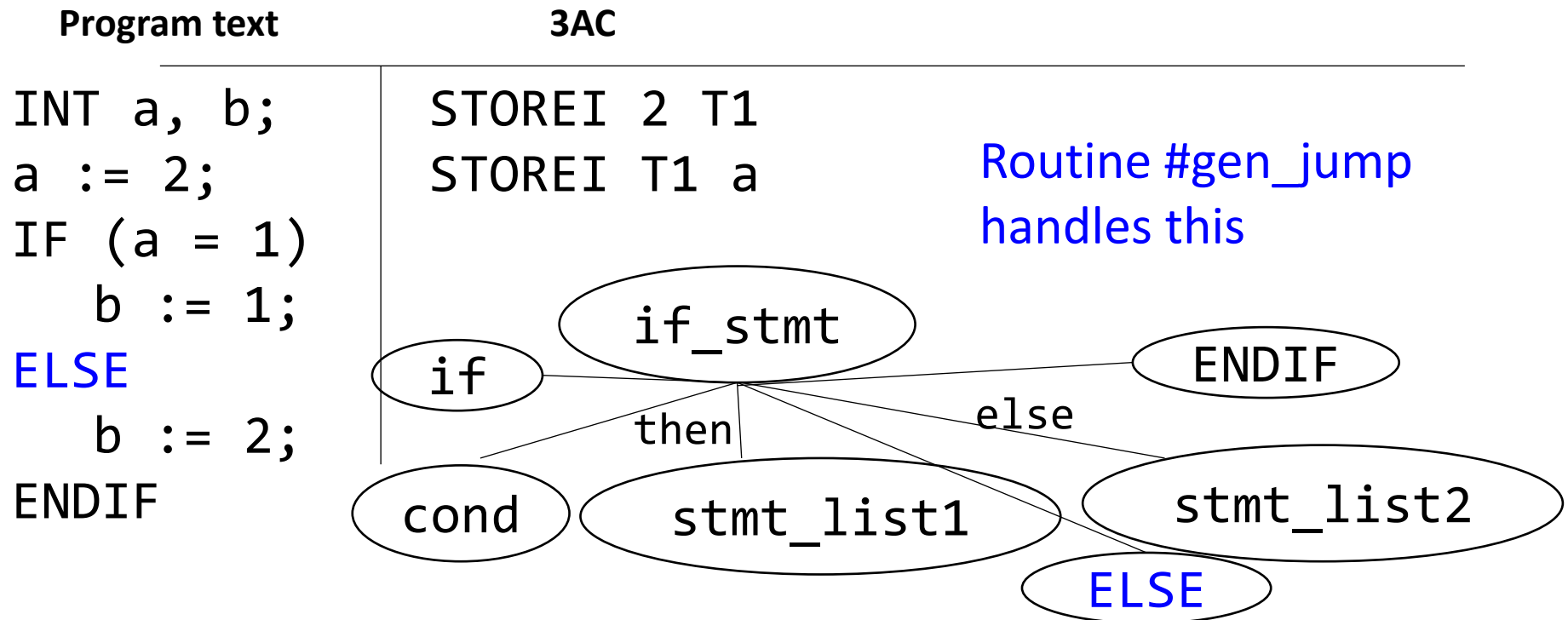
Code-generation – if-statement



3. Generate code for stmt_list1

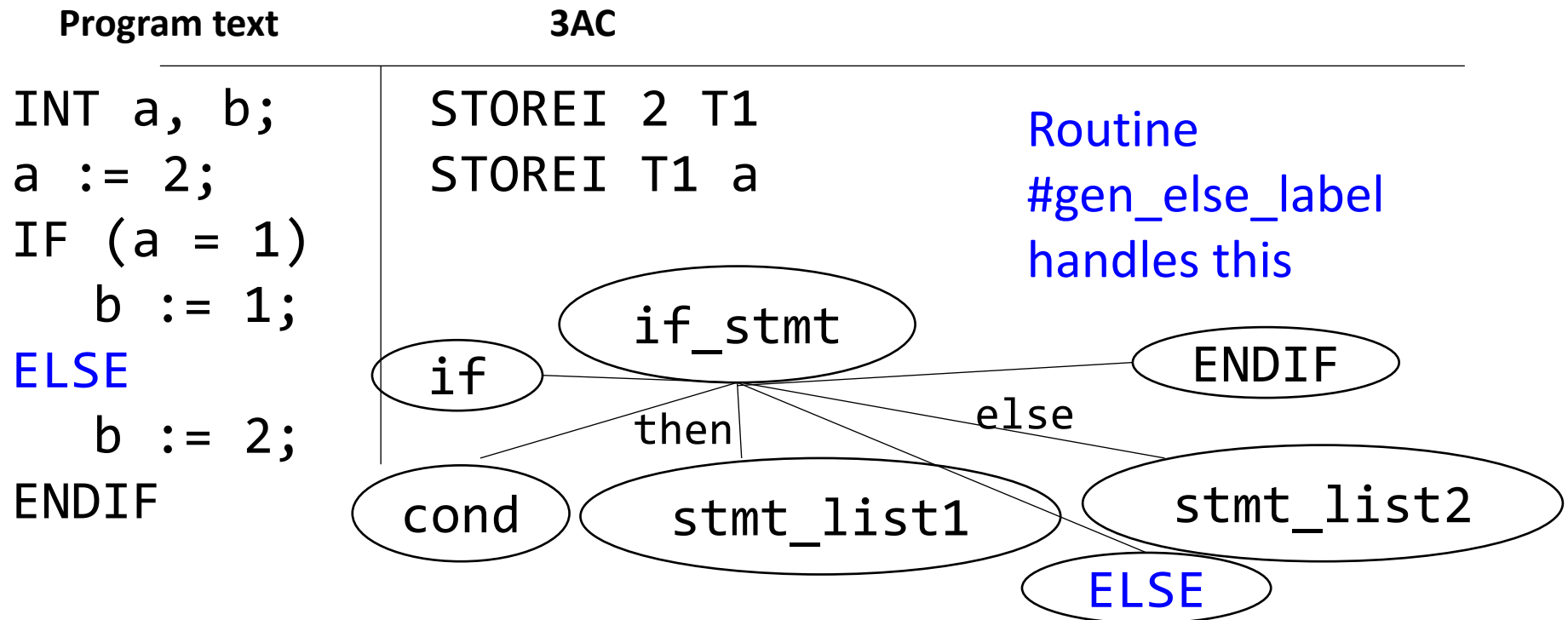
```
STOREI 1 T3  
STOREI T3 b
```

Code-generation – if-statement



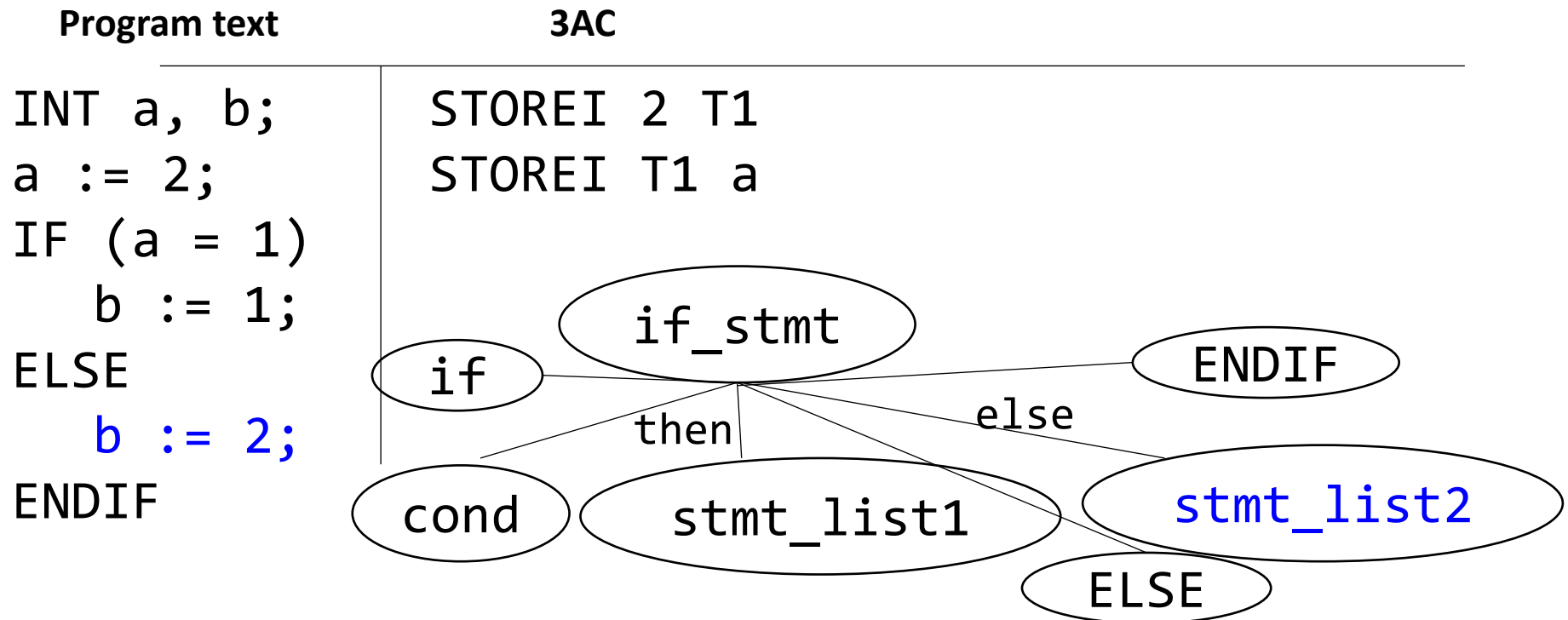
4. Generate unconditional jump to out label (label1).
JUMP label1

Code-generation – if-statement



5. Associate else part label (label12) with address of next instruction i.e. generate a statement: LABEL label12

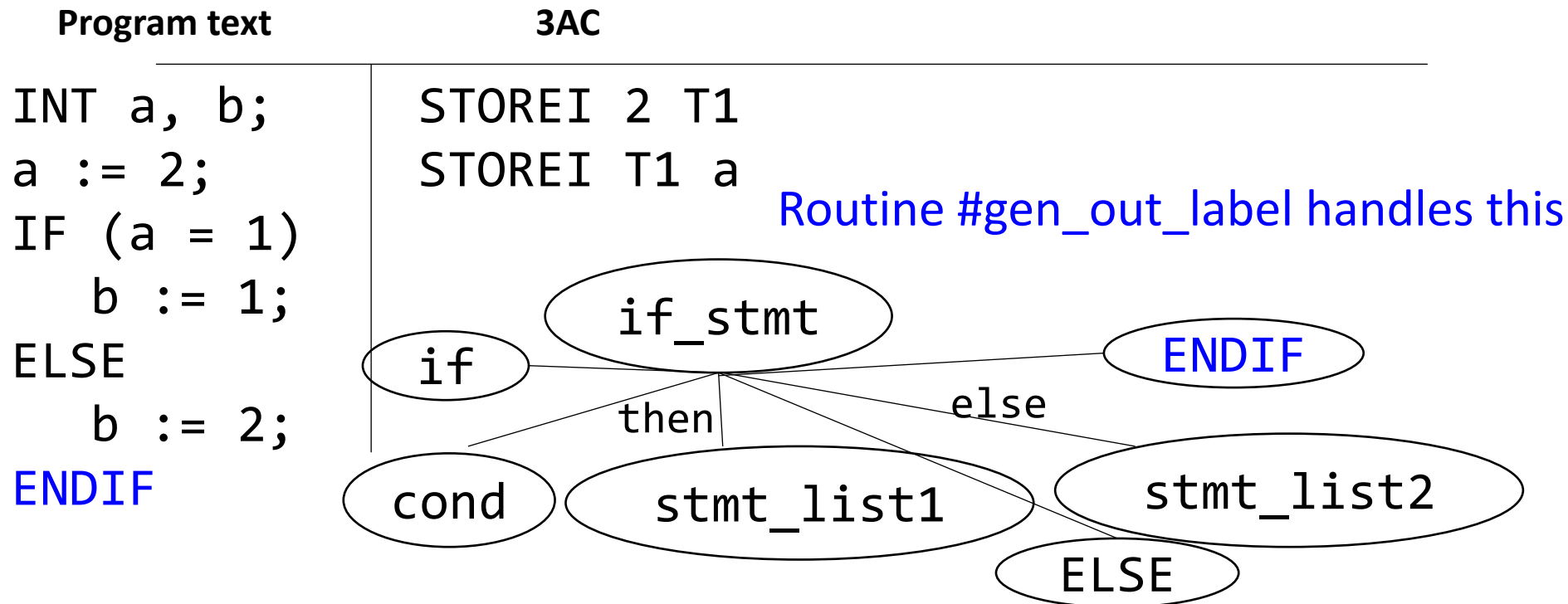
Code-generation – if-statement



5. Generate code for `stmt_list2`

```
STOREI 2 T4  
STOREI T4 b
```

Code-generation – if-statement



5. Associate out label (`label1`) with address of next instruction
i.e. generate a statement: `LABEL label1`

Observations

- We added semantic actions with tokens IF, ELSE, ENDIF
- Generated code is equivalent but not exact
 - e.g. “NE a T2 label1” is replaced with an equivalent “JUMPO bool_expr label1”
- Done in one pass ?

Will this approach work when generating machine code directly?

If construct with semantic actions

- If_stmt → if #start_if <b_expr> #testif then
 <stmt_list> <optional_elseif_part> <else_part>
 endif; #gen_out_label
- <optional_elseif_part> → elseif #gen_jump
 #gen_else_label <b_expr> #testif then <stmts>
- Else_part → else #gen_jump #gen_else_label
 <stmt_list>

Exercise: augment the grammar rule to handle elsif blocks.

Code-generation – if-statement

Program text	3AC
INT a, b;	STOREI 2 T1 //a := 2
a := 2;	STOREI T1 a
IF (a = 1)	STOREI 1 T2 //a = 1?
b := 1;	NE a T2 label1
ELIF (TRUE)	STOREI 1 T3 //b := 1
b := 2;	STOREI T3 b
ENDIF	JUMP label2 //to out label
	LABEL label1 //elsif label
	STOREI 1 T4 //TRUE can be handled by checking 1 = 1?
	STOREI 1 T5
	NE T4 T5 label3 //jump to the next elsif label
	STOREI 2 T6 //b := 2
	STOREI T6 b
	JUMP label2 //jump to out label
	LABEL label3 //out label
	LABEL label2 //out label

do-while

- `do{S}while(B);` //S is executed at least once and again and again and again... while B remains true

do-while

- `do{S}while(B);` //S is executed at least once and again and again and again... while B remains true

```
do #beginloop {S} while(B) #testloop ; #endloop
```

LOOP:

```
<stmt_list>
```

```
<bool_expr>
```

```
j<!op> OUT
```

```
jmp LOOP
```

OUT:

```
#beginloop
```

```
create labels LOOP and OUT
```

```
generate LABEL LOOP
```

```
#testloop
```

```
Check if the conditional statement B  
has the correct type (Boolean)
```

```
#endloop
```

```
Generate JUMP0 OUT
```

```
Generate JUMP LOOP
```


repeat-until

- `repeat{S}until(B);` //S is executed at least once and again and again and again... while B remains false

repeat-until

- `repeat{S}until(B);` //S is executed at least once and again and again and again... while B remains false

LOOP:

 <stmt_list>

 <bool_expr>

 j<!op> LOOP

OUT:

Generating code: for loops

```
for (<init_stmt>; <bool_expr>; <incr_stmt>)  
    <stmt_list>  
end
```



```
<init_stmt>  
LOOP:  
    <bool_expr>  
    j<!op> OUT  
    <stmt_list>  
INCR:  
    <incr_stmt>  
    jmp LOOP  
OUT:
```

- Execute init_stmt first
- Jump out of loop if bool_expr is false
- Execute incr_stmt after block, jump back to top of loop
- Question: Why do we have the INCR label?

Switch statements

```
switch (<expr>)  
  case <const_list>: <stmt_list>  
  case <const_list>: <stmt_list>  
  ...  
  default: <stmt_list>  
end
```

- Generated code should evaluate <expr> and make sure that some case matches the result
- Question: how to decide where to jump?

Deciding where to jump

- Problem: do not know *which label* to jump to until switch expression is evaluated
- Use a jump table: an array indexed by case values, contains address to jump to
 - If table is not full (i.e., some possible values are skipped), can point to a default clause
 - If default clause does not exist, this can point to error code
- Problems
 - If table is sparse, wastes a lot of space
 - If many choices, table will be very large

Jump table example

Consider the code:
((xxxx) is address of code)

Case x is
(0010) When 0: stmts
(0017) When 1: stmts
(0192) When 2: stmts
(0198) When 3 stmts;
(1000) When 5 stmts;
(1050) Else stmts;

Table only has one
Unnecessary row
(for choice 4)

Jump table has 6 entries:

0	JUMP 0010
1	JUMP 0017
2	JUMP 0192
3	JUMP 0198
4	JUMP 1050
5	JUMP 1000

Jump table example

Consider the code:
((xxxx) Is address of code)

Case x is
(0010) When 0: stmts0
(0017) When 1: stmts1
(0192) When 2: stmts2
(0198) When 3: stmts3
(1000) When 987: stmts4
(1050) When others: stmts5

Table only has 983 unnecessary rows.
Doesn't appear to be the right thing to do! **NOTE: table size is proportional to range of choice clauses, not number of clauses!**

Jump table has 6 entries:

0	JUMP 0010
1	JUMP 0017
2	JUMP 0192
3	JUMP 0198
4	JUMP 1050
...	JUMP 1050
986	JUMP 1050
987	JUMP 1000

Linear search example

Consider the code:

(xxxx) Is offset of local
Code start from the
Jump instruction

Case x is

(0010) When 0: stmts

(0017) When 1: stmts

(0192) When 2: stmts

(1050) When others stmts;

If there are a small number of choices, then do an in-line linear search. A straightforward way to do this is generate code analogous to an IF THEN ELSE.

If (x == 0) then stmts1;

Elseif (x = 1) then stmts2;

Elseif (x = 2) then stmts3;

Else stmts4;

$O(n)$ time, n is the size of the table, for each jump.

Dealing with jump tables

```
switch (<expr>)  
  case <const_list>: <stmt_list>  
  case <const_list>: <stmt_list>  
  ...  
  default: <stmt_list>  
end
```

```
    <expr>  
    <code for jump table>  
LABEL0:  
    <stmt_list>  
LABEL1:  
    <stmt_list>  
...  
DEFAULT:  
    <stmt_list>  
OUT:
```

- Generate labels, code, then build jump table
- Put jump table after generated code
- Why do we need the OUT label?
- In case of break statements